

LAPORAN AKHIR
PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

Disusun untuk memenuhi salah satu syarat kelulusan Mata Muliah
Praktikum Algoritma dan Struktur Data pada Program Studi S1 Sistem Informasi
Fakultas Ilmu Komputer Universitas Singaperbangsa Karawang
Semester Genap Tahun Akademik 2025/2026



Oleh:
Kelompok 3

Talitha Nailal Husna	2510631250050
Firman Hidayat	2510631250032
Radhin Ramadhan Hendestian	2510631250005

PROGRAM STUDI S1 SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG
2025/2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya. Laporan Akhir Praktikum yang berjudul “Laporan Akhir Praktikum Algoritma dan Struktur Data” ini dapat diselesaikan dengan baik dan tepat pada waktunya.

Laporan Akhir Praktikum Algoritma dan Struktur Data ini disusun untuk memenuhi salah satu syarat kelulusan Mata Kuliah Algoritma dan Struktur Data pada Program Studi S1 Sistem Informasi, Fakultas Ilmu Komputer, Universitas Singaperbangsa Karawang Semester Genap Tahun Akademik 2025/2026.

Dalam proses penyusunan laporan ini, kami memperoleh banyak bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu, melalui kesempatan ini kami ingin menyampaikan rasa terima kasih kepada:

1. Bapak **Taufik Ridwan, M.T.**, selaku dosen pengampu Mata Kuliah Algoritma dan Struktur Data yang senantiasa memberikan arahan, bimbingan, serta dukungan selama kegiatan praktikum dan penyusunan laporan ini.
2. Rekan-rekan mahasiswa Program Studi S1 Sistem Informasi yang telah memberikan semangat, bantuan, serta kerja sama selama proses praktikum hingga penyusunan laporan ini berlangsung.

Kami menyadari bahwa Laporan Akhir Praktikum Algoritma dan Struktur Data ini masih jauh dari kata sempurna dan masih terdapat berbagai kekurangan. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang bersifat membangun demi perbaikan dan penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, kami berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca, khususnya dalam bidang algoritma dan struktur data. Selain itu, laporan ini diharapkan dapat menjadi sarana pengembangan pengetahuan, pemahaman, serta kemampuan analisis dalam penerapan konsep algoritma dan struktur data secara sistematis, logis, dan efisien di dunia akademik maupun dalam perkembangan teknologi informasi di masa mendatang.

Karawang, 26 Mei 2026

Hormat Kami,

Kelompok 3

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	4
BAB II ANALISIS KEBUTUHAN	6
2.1 Analisis Sistem	6
2.2 Analisis Kebutuhan Fungsional.....	6
2.2.1 Fitur Login Sistem	7
2.2.2 Fitur Tambah Data Mahasiswa	7
2.2.3 Fitur Tampilkan Data.....	8
2.2.4 Fitur Edit Data	8
2.2.5 Fitur Hapus & Restore Data.....	9
2.2.6 Fitur Pencarian Data (Searching).....	9
2.2.7 Fitur Pengurutan Data (Sorting)	9
2.2.8 Fitur Statistik Nilai	9
2.2.9 Fitur Penyimpanan dan Pemuatan Data	10
2.3 Analisis Kebutuhan Non-Fungsional.....	10
2.4 Analisis Struktur.....	14
2.4.1 Struktur Data Mahasiswa.....	14
2.4.2 Struktur Data Array	16
2.4.3 Struktur Algoritma (Searching)	16
2.4.4 Struktur Algoritma (Sorting)	17
2.5 Analisis Alur Sistem.....	17
BAB III IMPLEMENTASI.....	20
3.1 Implementasi Program.....	20
3.1.1 Class Mahasiswa.....	20
3.1.2 Struktur Data.....	21
3.2 Implementasi Fitur Program.....	22
3.2.1 Tambah Data Mahasiswa.....	22
3.2.2 Edit Data	25
3.2.3 Hapus dan Restore Data.....	26
3.3 Implementasi Searching	28
3.3.1 Sequential Search.....	28
3.3.2 Binary Search	30
3.4 Implementasi Sorting.....	31
3.5 Implementasi Statistik	32
3.6 Implementasi File Handling	33
3.6.1 Simpan Data	33
3.6.2 Load Data	33

3.7 Implementasi Login dan Admin	34
3.7.1 Admin	35
3.7.2 User	36
3.8 Implementasi Interface	37
BAB IV ANALISIS KOMPLEKSITAS	38
4.1 Pendahuluan	38
4.2 Analisis Kompleksitas Waktu	38
4.2.1 Proses Login	39
4.2.2 Proses Tambah Data	39
4.2.3 Proses Edit Data	40
4.2.4 Proses Hapus Data	40
4.2.5 Proses Pencarian Nama	40
4.2.6 Proses Pencarian ID	41
4.2.7 Proses Pengurutan Data	41
4.2.8 Proses Statistik	41
4.2.9 Proses Simpan Data	42
4.3 Analisis Kompleksitas Ruang	42
4.3.1 Struktur Data Array	42
4.3.2 Quick Sort	42
4.3.3 Sequential Search dan Binary Search	43
4.3.4 Penyimpanan File	43
4.4 Tabel Analisis Kompleksitas	43
BAB V PENGUJIAN	46
5.1 Skenario Pengujian	46
5.1.1 Skenario 1: Login akun user dan admin	46
5.1.2 Skenario 2: Tampilkan data	47
5.1.3 Skenario 3: Tambah data mahasiswa	47
5.1.4 Skenario 4: Menambah data dengan ID duplikat	48
5.1.5 Skenario 5: Edit data	48
5.1.6 Skenario 6: Hapus dan restore data mahasiswa	48
5.1.7 Skenario 7: Pencarian nama mahasiswa	50
5.1.8 Skenario 8: Pencarian dengan ID mahasiswa	50
5.1.9 Skenario 9: Pencarian dengan Prodi mahasiswa	51
5.1.10 Skenario 10: Sorting By ID	52
5.1.11 Skenario 11: Sorting By Nama	52
5.1.12 Skenario 12: Sorting By Nilai	53
5.1.13 Skenario 13: Tabel Statistik	54
5.1.14 Skenario 14: Logout dan login kembali	55
BAB VI PENUTUP	56
6.1 Kesimpulan	56
6.2 Saran	57

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi pada era digital telah membawa perubahan besar dalam berbagai aspek kehidupan, termasuk pada bidang pendidikan dan pengelolaan data akademik. Pemanfaatan sistem berbasis komputer menjadi solusi penting dalam meningkatkan efektivitas, efisiensi, serta akurasi pengolahan data dibandingkan metode konvensional yang masih dilakukan secara manual. Dalam lingkungan perguruan tinggi, pengelolaan data mahasiswa khususnya data nilai akademik merupakan bagian yang sangat penting karena berkaitan langsung dengan proses evaluasi hasil belajar mahasiswa. Oleh sebab itu, diperlukan suatu sistem yang mampu membantu proses pengelolaan nilai secara cepat, tepat, dan terstruktur agar meminimalisir terjadinya kesalahan pencatatan maupun kehilangan data.

Algoritma dan Struktur Data merupakan salah satu mata kuliah fundamental dalam bidang ilmu komputer yang mempelajari cara penyusunan langkah-langkah logis (algoritma) serta pengorganisasian data (struktur data) untuk menyelesaikan suatu permasalahan secara efisien. Menurut Imilda, Suryadi, dan Ahmad (2024), penerapan algoritma dan struktur data dalam sistem informasi mampu meningkatkan efektivitas pengolahan data serta efisiensi proses administrasi berbasis komputer. Penguasaan konsep algoritma tidak hanya terbatas pada pemahaman teori, tetapi juga bagaimana konsep tersebut diimplementasikan ke dalam suatu program nyata yang dapat menyelesaikan permasalahan sehari-hari secara sistematis dan efisien. Oleh karena itu, mahasiswa dituntut untuk mampu mengembangkan sistem sederhana berbasis pemrograman sebagai bentuk implementasi langsung dari materi yang telah dipelajari.

Salah satu implementasi nyata dari penerapan algoritma dan struktur data adalah pembuatan Sistem Manajemen Nilai Mahasiswa berbasis bahasa pemrograman Java. Sistem ini dirancang untuk membantu proses pengelolaan data mahasiswa meliputi penambahan data, pengeditan data, penghapusan data, pencarian data, pengurutan data, hingga analisis statistik nilai mahasiswa secara otomatis. Dalam pengembangannya, sistem memanfaatkan berbagai konsep algoritma seperti *Sequential Search*, *Binary Search*, dan *Quick Sort* untuk meningkatkan efisiensi proses pencarian serta pengurutan data. Selain itu, struktur data *array* digunakan sebagai media penyimpanan utama data mahasiswa yang memungkinkan proses manipulasi data dilakukan secara terstruktur dan sistematis.

Perkembangan kebutuhan industri terhadap sistem informasi yang cepat dan akurat turut mendorong pentingnya penguasaan kemampuan pemrograman berbasis logika algoritmik. Menurut Goodrich, Tamassia, dan Goldwasser (2014), algoritma pencarian (*searching*) dan pengurutan (*sorting*) merupakan konsep fundamental dalam struktur data yang berperan penting dalam meningkatkan efisiensi sistem komputer. Oleh karena itu, melalui proyek akhir mata kuliah Algoritma dan Struktur Data ini, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga memperoleh pengalaman langsung dalam merancang dan mengembangkan sistem berbasis komputer yang mampu menyelesaikan permasalahan pengelolaan data secara nyata.

Sistem Manajemen Nilai Mahasiswa yang dikembangkan dalam proyek ini memiliki beberapa fitur utama seperti login admin dan user, penambahan data mahasiswa, pengeditan data, penghapusan dan restore data, pencarian data berdasarkan nama, ID, dan program studi, pengurutan data berdasarkan ID, nama, maupun nilai akhir, serta fitur statistik nilai mahasiswa. Sistem juga dilengkapi mekanisme penyimpanan data otomatis menggunakan file eksternal sehingga data tetap tersimpan meskipun program ditutup.

Dengan adanya fitur-fitur tersebut, sistem diharapkan mampu meningkatkan efektivitas pengolahan data akademik dibandingkan metode manual yang rentan terhadap kesalahan dan keterlambatan proses.

Dalam proses pengembangan sistem, terdapat beberapa permasalahan umum yang sering ditemui, seperti keterbatasan pemahaman mahasiswa dalam mengimplementasikan algoritma *searching* dan *sorting*, kesulitan dalam mengelola struktur data *array* secara dinamis, serta kurangnya pemahaman mengenai validasi input dan pengelolaan file pada bahasa pemrograman Java. Selain itu, efisiensi algoritma juga menjadi perhatian penting karena berkaitan dengan performa program ketika mengolah data dalam jumlah besar. Oleh sebab itu, diperlukan analisis kompleksitas algoritma untuk mengetahui tingkat efisiensi waktu dan ruang dari setiap metode yang digunakan dalam sistem. Adapun beberapa permasalahan umum yang ditemukan dalam pengembangan Sistem Manajemen Nilai Mahasiswa dapat dirangkum pada tabel berikut:

Tabel 1.1 Permasalahan Umum

No	Permasalahan Umum
1	Pengolahan data mahasiswa masih dilakukan secara manual sehingga rentan terjadi kesalahan pencatatan
2	Proses pencarian data membutuhkan waktu yang cukup lama apabila jumlah data banyak
3	Belum adanya sistem otomatis untuk pengurutan dan analisis nilai mahasiswa
4	Kurangnya pemahaman implementasi algoritma <i>searching</i> dan <i>sorting</i> pada program nyata
5	Risiko kehilangan data karena belum adanya mekanisme penyimpanan otomatis
6	Validasi input data mahasiswa masih sering menyebabkan kesalahan program

Sumber: (Kelompok 3, 2026)

Pengembangan Sistem Manajemen Nilai Mahasiswa ini diharapkan mampu menjadi sarana pembelajaran implementatif bagi

mahasiswa dalam memahami konsep algoritma dan struktur data secara lebih mendalam. Sistem ini tidak hanya berfungsi sebagai media latihan pemrograman, tetapi juga menjadi bentuk penerapan nyata dari konsep *searching*, *sorting*, validasi data, serta pengolahan *file* yang banyak digunakan dalam pengembangan sistem informasi modern. Dengan demikian, proyek ini diharapkan mampu meningkatkan kemampuan analitis, logika pemrograman, dan keterampilan *problem solving* mahasiswa dalam menghadapi tantangan perkembangan teknologi informasi yang semakin kompleks.

1.2 Tujuan Praktikum

Pembuatan Sistem Manajemen Nilai Mahasiswa pada proyek akhir mata kuliah Algoritma dan Struktur Data ini memiliki tujuan untuk mengimplementasikan konsep algoritma dan struktur data ke dalam sebuah program berbasis Java yang mampu mengelola data mahasiswa secara efektif dan efisien. Adapun tujuan dari pengembangan sistem ini adalah sebagai berikut:

1. Mengimplementasikan konsep struktur data *array* dalam penyimpanan dan pengelolaan data mahasiswa secara terstruktur.
2. Mengembangkan sistem pengelolaan nilai mahasiswa berbasis Java yang mampu melakukan proses tambah, edit, hapus, restore, dan tampil data secara otomatis.
3. Mengimplementasikan algoritma *Sequential Search* dan *Binary Search* untuk meningkatkan efisiensi proses pencarian data mahasiswa.
4. Mengimplementasikan algoritma *Quick Sort* dalam proses pengurutan data berdasarkan ID, nama, dan nilai akhir mahasiswa.
5. Menerapkan validasi input data untuk meminimalisir kesalahan pengguna dalam proses pengolahan data.

6. Mengembangkan fitur statistik nilai mahasiswa untuk mengetahui rata-rata nilai, distribusi grade, serta persentase kelulusan mahasiswa.
7. Menerapkan mekanisme penyimpanan data otomatis menggunakan file eksternal agar data tetap tersimpan dengan aman.
8. Meningkatkan kemampuan mahasiswa dalam memahami implementasi algoritma dan struktur data melalui pengembangan program berbasis studi kasus nyata.
9. Melatih kemampuan problem solving, logika pemrograman, dan analisis kompleksitas algoritma dalam pengembangan sistem informasi sederhana.
10. Menghasilkan sistem pengelolaan nilai mahasiswa yang efektif, efisien, dan mudah digunakan sebagai media pembelajaran implementasi algoritma dan struktur data.

BAB II

ANALISIS KEBUTUHAN

2.1 Analisis Sistem

Sistem Manajemen Nilai Mahasiswa merupakan program berbasis *console* yang dikembangkan menggunakan bahasa pemrograman Java untuk membantu proses pengelolaan data nilai mahasiswa secara terstruktur, cepat, dan efisien. Sistem ini dirancang untuk memenuhi kebutuhan administrasi akademik sederhana yang meliputi pengolahan data mahasiswa, penginputan nilai, pencarian data, pengurutan data, hingga penyimpanan data secara permanen menggunakan media penyimpanan file.

Pengembangan sistem ini memanfaatkan konsep algoritma dan struktur data sebagai fondasi utama dalam proses pengolahan data. Menurut Lafore (2017), struktur data merupakan metode pengorganisasian data yang digunakan agar proses pengolahan, pencarian, dan manipulasi data dapat dilakukan secara efisien. Selain itu, implementasi algoritma pencarian (*searching*) dan pengurutan (*sorting*) berperan penting dalam meningkatkan performa sistem informasi berbasis komputer.

Pada sistem yang dikembangkan, data mahasiswa disimpan menggunakan struktur data array bertipe objek Mahasiswa[], sedangkan proses pengolahan data memanfaatkan algoritma *Sequential Search*, *Binary Search*, dan *Quick Sort*. Pemanfaatan algoritma tersebut bertujuan untuk meningkatkan efisiensi pengolahan data mahasiswa dalam jumlah besar.

2.2 Analisis Kebutuhan Fungsional

Kebutuhan fungsional merupakan kebutuhan yang menjelaskan layanan atau fitur yang harus dimiliki oleh sistem agar dapat berjalan sesuai tujuan pengembangan. Berdasarkan kode

program yang telah dikembangkan, sistem memiliki beberapa fitur utama sebagai berikut.

2.2.1 Fitur Login Sistem

Fitur login digunakan sebagai mekanisme keamanan sistem untuk membedakan hak akses antara administrator dan pengguna (user). Sistem menyediakan dua jenis akun, yaitu:

1. Admin
2. Mahasiswa (*user*)

Admin memiliki akses penuh terhadap seluruh fitur sistem, sedangkan mahasiswa hanya dapat melihat data nilainya sendiri.

Fungsi:

1. Memvalidasi username dan password.
2. Menentukan hak akses pengguna.
3. Menampilkan menu sesuai peran pengguna.

2.2.2 Fitur Tambah Data Mahasiswa

Fitur ini digunakan untuk menambahkan data mahasiswa baru ke dalam sistem. Data yang diinput:

- ID Mahasiswa
- Nama Mahasiswa
- Program Studi
- Nilai Tugas
- Nilai UTS
- Nilai UAS

Proses Sistem:

- Melakukan validasi ID agar tidak duplikat.
- Menghitung nilai akhir otomatis.
- Menentukan grade dan status kelulusan.
- Menyimpan data ke *array*.

2.2.3 Fitur Tampilkan Data

Fitur ini berfungsi untuk menampilkan seluruh data mahasiswa yang aktif di dalam sistem.

Fungsi:

- Menampilkan data dalam bentuk tabel.
- Menampilkan status kelulusan dengan pewarnaan terminal.
- Menampilkan data sesuai hak akses pengguna.

Informasi yang ditampilkan:

- ID
- Nama
- Program Studi
- Nilai Tugas
- Nilai UTS
- Nilai UAS
- Nilai Akhir
- Grade
- Status Kelulusan

2.2.4 Fitur Edit Data

Fitur edit digunakan untuk memperbarui data mahasiswa yang telah tersimpan.

Data yang dapat diubah:

- Nama
- Program Studi
- Nilai

Fungsi:

- Mempermudah koreksi data.
- Menghitung ulang nilai akhir secara otomatis.

2.2.5 Fitur Hapus & Restore Data

Sistem menerapkan konsep *soft delete*, yaitu data tidak dihapus permanen melainkan hanya dinonaktifkan menggunakan variabel boolean aktif.

Fungsi:

- Menghapus data sementara.
- Mengembalikan data yang telah dihapus.

2.2.6 Fitur Pencarian Data (*Searching*)

Sistem menyediakan beberapa metode pencarian data.

- a. Pencarian Nama (Cari Nama): Menggunakan algoritma *Sequential Search*.
- b. Pencarian ID (Cari ID): Menggunakan algoritma Binary Search setelah data diurutkan berdasarkan ID
- c. Pencarian Program Studi (Cari Prodi): Menggunakan algoritma *Sequential Search*.

2.2.7 Fitur Pengurutan Data (*Sorting*)

Sistem menggunakan algoritma *Quick Sort* untuk melakukan pengurutan data.

Jenis Pengurutan:

- *Sort* berdasarkan ID
- *Sort* berdasarkan Nama
- *Sort* berdasarkan Nilai Akhir

2.2.8 Fitur Statistik Nilai

Fitur statistik digunakan untuk menampilkan ringkasan data akademik mahasiswa.

Informasi Statistik:

- Total mahasiswa
- Jumlah lulus

- Jumlah tidak lulus
- Rata-rata nilai
- Persentase kelulusan
- Distribusi grade

Fungsi:

- Membantu analisis performa akademik mahasiswa.
- Mempermudah evaluasi hasil pembelajaran.

2.2.9 Fitur Penyimpanan dan Pemuatan Data

Sistem menggunakan file data.txt sebagai media penyimpanan permanen.

Fungsi:

- Menyimpan data otomatis (*auto save*).
- Memuat data saat sistem dijalankan kembali.

2.3 Analisis Kebutuhan Non-Fungsional

Kebutuhan non-fungsional merupakan kebutuhan pendukung yang menentukan kualitas sistem, performa, keamanan, serta kenyamanan penggunaan program. Kebutuhan ini tidak secara langsung berkaitan dengan proses utama sistem, tetapi sangat berpengaruh terhadap efektivitas dan keberhasilan implementasi program. Pada Sistem Manajemen Nilai Mahasiswa berbasis Java yang dikembangkan, kebutuhan non-fungsional dijelaskan secara rinci sebagai berikut:

Tabel 1.2 Analisis Kebutuhan Non-Fungsional

No	Kebutuhan Non-Fungsional	Penjelasan
----	--------------------------	------------

1	Bahasa Pemrograman	Sistem dikembangkan menggunakan bahasa pemrograman Java karena Java memiliki sifat <i>platform independent</i> , sehingga program dapat dijalankan di berbagai sistem operasi seperti Windows, Linux, maupun macOS selama tersedia Java Runtime Environment (JRE). Selain itu, Java mendukung konsep <i>Object-Oriented Programming</i> (OOP) yang memudahkan pengelolaan kode program menjadi lebih terstruktur dan modular.
2	Media Penyimpanan Data	Sistem menggunakan file berekstensi .txt sebagai media penyimpanan data mahasiswa secara permanen. Pemilihan file teks dilakukan karena implementasinya sederhana, ringan, serta mudah dipahami dalam pembelajaran algoritma dan struktur data. Data yang tersimpan meliputi identitas mahasiswa, nilai, grade, dan status kelulusan sehingga data tetap tersedia meskipun program ditutup.
3	Antarmuka Sistem	Program menggunakan antarmuka berbasis <i>console</i> atau terminal. Antarmuka ini dipilih karena lebih sederhana dan sesuai untuk pembelajaran dasar algoritma serta struktur data tanpa memerlukan pengembangan tampilan grafis (<i>Graphical User Interface</i> / GUI). Melalui antarmuka console, pengguna

		dapat berinteraksi dengan sistem menggunakan menu numerik dan input keyboard.
4	Kapasitas Penyimpanan Data	Sistem dirancang untuk mampu menyimpan maksimal 200 data mahasiswa menggunakan struktur data array. Pembatasan kapasitas ini disesuaikan dengan penggunaan array statis pada program Java. Walaupun kapasitasnya terbatas, jumlah tersebut sudah cukup untuk kebutuhan simulasi praktikum dan pembelajaran pengolahan data mahasiswa dalam skala kecil hingga menengah.
5	Keamanan Sistem	Sistem memiliki fitur login untuk membatasi akses pengguna. Terdapat dua jenis hak akses yaitu administrator dan mahasiswa (<i>user</i>). Administrator dapat mengelola seluruh data mahasiswa, sedangkan mahasiswa hanya dapat melihat data miliknya sendiri. Mekanisme ini bertujuan menjaga keamanan serta mencegah manipulasi data oleh pihak yang tidak memiliki hak akses.
6	Validasi Input Data	Sistem menerapkan validasi input untuk mengurangi kesalahan pengisian data. Contohnya, nilai mahasiswa hanya dapat diinput dalam rentang tertentu (0–100), dan ID mahasiswa tidak boleh kosong atau duplikat. Validasi ini penting untuk menjaga konsistensi, akurasi, dan

		integritas data yang tersimpan di dalam sistem.
7	Kinerja Sistem	Sistem dirancang agar mampu menjalankan proses pencarian, pengurutan, dan pengolahan data secara cepat serta efisien. Untuk mendukung performa tersebut, program menggunakan algoritma <i>Quick Sort</i> , <i>Sequential Search</i> , dan <i>Binary Search</i> sehingga proses manipulasi data dapat dilakukan dengan waktu yang relatif singkat meskipun jumlah data bertambah.
8	Portabilitas Sistem	Program memiliki tingkat portabilitas yang tinggi karena dapat dijalankan pada berbagai perangkat dan sistem operasi tanpa perlu perubahan kode program yang signifikan. Selama perangkat memiliki compiler Java atau JRE, sistem dapat dijalankan dengan baik.
9	Kemudahan Penggunaan (Usability)	Sistem dirancang dengan menu yang sederhana dan mudah dipahami oleh pengguna, terutama mahasiswa yang baru mempelajari algoritma dan struktur data. Setiap fitur disusun secara sistematis sehingga pengguna dapat mengoperasikan program tanpa memerlukan pelatihan khusus.
10	Pemeliharaan Sistem (Maintainability)	Struktur kode program dibuat secara modular menggunakan metode (<i>method</i>) dan class sehingga memudahkan proses pengembangan maupun perbaikan

		sistem di masa mendatang. Jika terdapat penambahan fitur baru, pengembang dapat melakukan modifikasi tanpa harus mengubah keseluruhan program.
--	--	--

Kebutuhan non-fungsional memiliki peran penting dalam mendukung kualitas keseluruhan sistem. Walaupun fitur utama sistem berfokus pada pengolahan data mahasiswa, aspek seperti keamanan, performa, kemudahan penggunaan, dan penyimpanan data menjadi faktor utama agar sistem dapat digunakan secara optimal. Dalam pengembangan sistem berbasis algoritma dan struktur data, kebutuhan non-fungsional juga membantu memastikan bahwa implementasi algoritma berjalan secara efisien serta mampu menangani proses pengolahan data dengan stabil dan terorganisasi.

Menurut Pressman dan Maxim (2020), kebutuhan non-fungsional berfungsi sebagai standar kualitas perangkat lunak yang mencakup aspek performa, keamanan, reliabilitas, dan kemudahan pemeliharaan sistem. Oleh karena itu, analisis kebutuhan non-fungsional menjadi bagian penting dalam proses pengembangan perangkat lunak agar sistem tidak hanya berjalan dengan benar, tetapi juga memiliki kualitas yang baik dan mudah digunakan.

2.4 Analisis Struktur

Struktur data merupakan metode penyimpanan dan pengorganisasian data agar proses akses dan manipulasi dapat dilakukan secara efisien

2.4.1 Struktur Data Mahasiswa

Program menggunakan *class* Mahasiswa sebagai representasi objek data mahasiswa.

Tabel 1.3 Tipe Data dari Atribut Mahasiswa

Atribut	Tipe Data	Fungsi
id	String	Menyimpan ID mahasiswa
nama	String	Menyimpan nama mahasiswa
prodi	String	Menyimpan program studi
tugas	double	Nilai tugas
uts	double	Nilai UTS
uas	double	Nilai UAS
akhir	double	Nilai akhir
grade	String	Grade mahasiswa
status	String	Status kelulusan
aktif	boolean	Status data aktif

Struktur data mahasiswa merupakan komponen inti dalam Sistem Manajemen Nilai Mahasiswa karena seluruh proses pengolahan data dilakukan berdasarkan objek Mahasiswa. Pada program yang dikembangkan, setiap mahasiswa direpresentasikan sebagai sebuah objek yang memiliki atribut identitas, nilai akademik, serta status kelulusan. Pendekatan ini menerapkan konsep *Object-Oriented Programming* (OOP), yaitu metode pemrograman yang mengorganisasikan data dan fungsi ke dalam sebuah objek agar program lebih terstruktur, mudah dikembangkan, dan efisien dalam pengelolaan data.

Penggunaan *class* Mahasiswa mempermudah sistem dalam melakukan berbagai proses seperti penambahan data, pencarian, pengurutan, pengeditan, hingga penyimpanan data. Seluruh atribut yang dimiliki mahasiswa disimpan dalam satu kesatuan objek sehingga proses manipulasi data menjadi lebih sederhana dan terorganisasi.

Selain itu, atribut seperti akhir, *grade*, dan status tidak hanya berfungsi sebagai tempat penyimpanan data, tetapi juga dihasilkan melalui proses perhitungan otomatis berdasarkan nilai tugas, UTS, dan UAS. Hal ini menunjukkan bahwa struktur data yang digunakan tidak hanya menyimpan informasi, melainkan juga mendukung proses logika program secara langsung.

Variabel aktif bertipe boolean digunakan untuk menerapkan konsep *soft delete*, yaitu metode penghapusan data tanpa benar-benar menghilangkan data dari memori atau file penyimpanan. Dengan pendekatan ini, data mahasiswa yang dihapus masih dapat dipulihkan kembali melalui fitur *restore* data. Teknik ini lebih aman dibanding penghapusan permanen karena dapat meminimalkan risiko kehilangan data secara tidak sengaja.

2.4.2 Struktur Data Array

Sistem menggunakan array objek: static Mahasiswa[]
data = new Mahasiswa[200];

Fungsi:

- Menyimpan seluruh data mahasiswa.
- Mempermudah proses iterasi dan pencarian.

Kelebihan:

- Implementasi sederhana.
- Akses data cepat menggunakan indeks.

Kekurangan:

- Ukuran array bersifat tetap.
- Kurang fleksibel untuk data besar.

2.4.3 Struktur Algoritma (*Searching*)

Sequential Search digunakan pada:

- Cari Nama
- Cari Prodi

Karakteristik:

- Mencari data satu per satu.
- Cocok untuk data kecil.

Binary Search digunakan pada:

- Cari ID

Karakteristik:

- Data harus terurut.
- Lebih cepat dibanding sequential search.

2.4.4 Struktur Algoritma (*Sorting*)

Program menggunakan algoritma *Quick Sort*.

Fungsi:

- Mengurutkan data berdasarkan:

- ID
- Nama
- Nilai

Mekanisme:

- Menggunakan teknik *divide and conquer*.
- Memilih pivot untuk membagi data.

Keunggulan:

- Cepat untuk data besar.
- Efisien dalam penggunaan memori.

2.5 Analisis Alur Sistem

Analisis alur sistem dilakukan untuk memahami bagaimana proses kerja Sistem Manajemen Nilai Mahasiswa berlangsung secara terstruktur mulai dari program dijalankan hingga sistem selesai digunakan. Analisis ini penting karena menunjukkan hubungan antarfitur, aliran data, proses pengolahan informasi, serta implementasi algoritma dan struktur data di dalam sistem.

Sistem dirancang menggunakan pendekatan berbasis menu (*menu-driven system*) sehingga pengguna dapat berinteraksi dengan program melalui pilihan menu yang tersedia pada terminal (*console*). Setiap proses di dalam sistem saling terhubung dan berjalan secara berurutan agar pengelolaan data mahasiswa dapat dilakukan secara efektif, akurat, dan sistematis.

Secara umum, alur sistem terdiri dari beberapa tahapan utama yaitu proses inisialisasi sistem, autentikasi pengguna, pengolahan data mahasiswa, manipulasi data menggunakan algoritma, hingga penyimpanan data permanen.

Alur utama Sistem Manajemen Nilai Mahasiswa dimulai ketika program dijalankan dan sistem melakukan inisialisasi data serta membaca data mahasiswa yang tersimpan pada file *data.txt*. Tahap ini bertujuan agar data sebelumnya tetap tersedia dan tidak hilang ketika program dijalankan kembali. Setelah itu, pengguna melakukan proses login menggunakan *username* dan *password* untuk menentukan hak akses sistem, baik sebagai administrator maupun mahasiswa (*user*).

Setelah login berhasil, sistem akan menampilkan menu utama yang berisi berbagai fitur pengolahan data mahasiswa. Pengguna kemudian dapat memilih fitur seperti menambah data, mengedit data, menghapus data, melakukan pencarian (*searching*), pengurutan (*sorting*), hingga melihat statistik nilai mahasiswa. Pada tahap ini, sistem memanfaatkan algoritma dan struktur data untuk memproses informasi secara efisien.

Dalam proses pencarian data, sistem menggunakan algoritma *Sequential Search* dan *Binary Search*, sedangkan proses pengurutan data menggunakan algoritma *Quick Sort*. Penggunaan algoritma tersebut membantu mempercepat proses pengolahan data serta meningkatkan efisiensi sistem.

Tahap terakhir adalah penyimpanan data secara otomatis ke dalam file sehingga seluruh perubahan data mahasiswa dapat

tersimpan secara permanen. Dengan alur sistem yang terstruktur, program mampu mengelola data mahasiswa secara efektif, sistematis, dan mudah digunakan.

Poin-Poin Penting Alur Sistem:

- Sistem dijalankan dan melakukan inisialisasi program.
- Data mahasiswa dimuat dari file penyimpanan.
- Pengguna melakukan login untuk mendapatkan hak akses.
- Sistem menampilkan menu utama sesuai pengguna.
- Pengguna memilih fitur pengolahan data.
- Sistem memproses data menggunakan algoritma dan struktur data.
- Data disimpan kembali secara otomatis ke file penyimpanan.

BAB III

IMPLEMENTASI

3.1 Implementasi Program

3.1.1 Class Mahasiswa

```
class Mahasiswa {
    String id, nama, prodi;
    double tugas, uts, uas, akhir;
    String grade, status;
    boolean aktif = true;
    boolean dihapus = false;

    Mahasiswa(String id, String nama, String prodi, double t, double u, double ua) {
        this.id = id;
        this.nama = nama;
        this.prodi = prodi;
        this.tugas = t;
        this.uts = u;
        this.uas = ua;
        hitung();
    }

    void hitung() {
        akhir = (tugas * 0.3) + (uts * 0.3) + (uas * 0.4);
        if (akhir >= 85) grade = "A";
        else if (akhir >= 75) grade = "B";
        else if (akhir >= 65) grade = "C";
        else if (akhir >= 50) grade = "D";
        else grade = "E";
        status = akhir >= 60 ? "LULUS" : "TIDAK LULUS";
    }
}
```

Program menggunakan class Mahasiswa sebagai *blueprint* atau template objek mahasiswa. Class ini menyimpan data:

- ID mahasiswa
- Nama mahasiswa
- Program studi
- Nilai tugas
- Nilai UTS
- Nilai UAS
- Nilai akhir
- Grade
- Status kelulusan

Selain itu terdapat method `hitung()` yang digunakan untuk menghitung nilai akhir mahasiswa dan menentukan grade secara otomatis

Rumus nilai akhir:

$$\text{NilaiAkhir} = (\text{Tugas}(30\%)) + (\text{UTS}(30\%)) + (\text{UAS}(40\%))$$

Penentuan grade:

- A : ≥ 85
- B : ≥ 75
- C : ≥ 65
- D : ≥ 50
- E : < 50

Status kelulusan:

- LULUS jika nilai akhir ≥ 60
- TIDAK LULUS jika nilai akhir < 60

3.1.2 Struktur Data

Struktur data utama yang digunakan adalah array of object:

```
static Mahasiswa[] data = new Mahasiswa[200];
```

Array tersebut digunakan untuk menyimpan maksimal 200 data mahasiswa dalam bentuk object. Variabel n digunakan untuk menghitung jumlah data yang tersimpan.

3.2 Implementasi Fitur Program

3.2.1 Tambah Data Mahasiswa

```
// ----- TAMBAH -----
static void tambah() {
    String terakhir = "-";
    for (int i = 0; i < n; i++) {
        if (data[i] != null && data[i].aktif) {
            if (terakhir.equals(anObject: "-") ||
                data[i].id.compareTo(terakhir) > 0) {
                terakhir = data[i].id;
            }
        }
    }

    System.out.println(CYAN +
        "ID terakhir yang digunakan: "
        + terakhir + RESET);

    // WAJIB ADA
    String id;

    while (true) {
        System.out.print(s: "Masukkan ID: ");
        id = in.nextLine();

        if (cekID(id)) {
            System.out.println(MERAH +
                "ID sudah ada! Gunakan ID lain."
                + RESET);
        } else {
            break;
        }
    }

    System.out.print(s: "Nama: ");
    String nama = in.nextLine();

    System.out.print(s: "Prodi: ");
    String prodi = in.nextLine();

    double t = inputNilai(namaNilai: "Tugas");
    double u = inputNilai(namaNilai: "UTS");
    double ua = inputNilai(namaNilai: "UAS");
    in.nextLine();

    data[n++] = new Mahasiswa(
        id, nama, prodi, t, u, ua
    );

    System.out.println(HIJAU +
        "Data berhasil ditambahkan!"
        + RESET);

    simpan();
}
```

Fitur tambah data digunakan untuk memasukkan data mahasiswa baru ke dalam sistem. Pada proses ini program melakukan

- validasi ID agar tidak duplicate,

```
// ----- CEK ID -----
static boolean cekID(String id) {
    for (int i = 0; i < n; i++) {
        if (data[i] != null &&
            data[i].aktif &&
            data[i].id.equalsIgnoreCase(id)) {
            return true;
        }
    }
    return false;
}

// ----- CEK ID AKTIF -----
static int cariIndexID(String id) {
    for (int i = 0; i < n; i++) {
        if (data[i].aktif &&
            data[i].id.equalsIgnoreCase(id)) {
            return i;
        }
    }
    return -1;
}
```

Jika ID sudah ada maka program menampilkan pesan:

"ID sudah ada! Gunakan ID lain."

- validasi nilai agar hanya menerima angka,

```
// ----- VALIDASI ANGKA -----
static int inputAngka(String teks, int min, int max) {
    int angka;
    while (true) {
        System.out.print(teks);

        if (in.hasNextInt()) {
            angka = in.nextInt();
            in.nextLine();

            if (angka >= min && angka <= max) {
                return angka;
            } else {
                System.out.println(MERAH +
                    "Input harus " + min +
                    " - " + max + "!" +
                    RESET);
            }
        } else {
            System.out.println(MERAH +
                "Input harus angka!" +
                RESET);
            in.nextLine();
        }
    }
}
```

Jika user memasukkan huruf pada nilai:

"Input harus angka!"

- validasi range nilai 1–100.

```
// ===== VALIDASI NILAI =====
static double inputNilai(String namaNilai) {

    double nilai;

    while (true) {

        System.out.print(namaNilai + ": ");

        // cek apakah input angka
        if (in.hasNextDouble()) {

            nilai = in.nextDouble();

            // cek range nilai
            if (nilai >= 1 && nilai <= 100) {

                return nilai;

            } else {

                System.out.println(MERAH +
                    "Nilai harus range 1 - 100!"
                    + RESET);

            }

        } else {

            System.out.println(MERAH +
                "Input harus angka!"
                + RESET);

            // membersihkan input salah
            in.next();

        }

    }

}
```

Jika nilai di luar range:

```
"Nilai harus range 1 - 100!"
```

Fitur ini menggunakan:

- array,
- percabangan,
- perulangan,
- validasi input,
- dan object mahasiswa.

3.2.2 Edit Data

```
// ----- EDIT -----
static void edit() {
    System.out.print(s: "ID: ");
    String id = in.next();

    int i = cariIndexID(id);
    if (i != -1) {
        if (data[i].id.equals(id)) {
            in.nextLine();

            System.out.print(s: "Ubah Nama? (ya/tidak): ");
            if (in.nextLine().equalsIgnoreCase(anotherString: "ya")) {
                System.out.print(s: "Nama baru: ");
                data[i].nama = in.nextLine();
            }

            System.out.print(s: "Ubah Prodi? (ya/tidak): ");
            if (in.nextLine().equalsIgnoreCase(anotherString: "ya")) {
                System.out.print(s: "Prodi baru: ");
                data[i].prodi = in.nextLine();
            }

            System.out.print(s: "Ubah Nilai? (ya/tidak): ");
            if (in.nextLine().equalsIgnoreCase(anotherString: "ya")) {
                data[i].tugas = inputNilai(namaNilai: "Tugas");
                data[i].uts = inputNilai(namaNilai: "UTS");
                data[i].uas = inputNilai(namaNilai: "UAS");

                in.nextLine();

                data[i].hitung();
            }
            simpan();
            return;
        }
    }
    else {
        System.out.println(MERAH +
            "ID tidak ditemukan!"
            + RESET);
    }
}
```

Fitur edit digunakan untuk mengubah data mahasiswa berdasarkan ID. Program akan mencari data menggunakan Sequential Search lalu menampilkan pilihan:

- ubah nama,
- ubah program studi,
- ubah nilai.

Jika nilai diubah maka sistem akan menghitung ulang nilai akhir dan grade mahasiswa secara otomatis.

3.2.3 Hapus dan Restore Data

```
// ----- HAPUS -----  
static void hapus() {  
    System.out.print(s: "ID: ");  
    String id = in.next();  
    int i = cariIndexID(id);  
    if (i != -1) {  
        if (data[i].id.equals(id)) {  
            data[i].aktif = false;  
            simpan();  
            System.out.println(x: "Data dihapus!");  
            return;  
        }  
    }  
    System.out.println(x: "Tidak ditemukan!");  
}
```

Fitur hapus digunakan untuk mengubah status data mahasiswa menjadi nonaktif dengan:

```
data[i].aktif = false;
```

Data sebenarnya tidak dihapus permanen sehingga dapat dikembalikan menggunakan fitur restore.

```
// ----- RESTORE -----
static void restore() {
    boolean ada = false;
    int no = 1;
    System.out.println(
        x: "\n"
    );

    System.out.printf(
        format: "%-53s \n",
        ...args: "DATA MAHASISWA TERHAPUS");

    System.out.println(
        x: " "
    );

    System.out.printf(
        format: "%-4s%-8s%-20s%-20s\n",
        ...args: "No", "ID", "Nama", "Program Studi");

    System.out.println(
        x: " "
    );

    for (int i = 0; i < n; i++) {
        if (!data[i].aktif) {
            ada = true;
            System.out.printf(
                format: "%-4d%-8s%-20s%-20s\n",
                no++,
                data[i].id,
                data[i].nama,
                data[i].prodi);
        }
    }

    if (!ada) {
        System.out.printf(
            format: "%-53s \n",
            ...args: "TIDAK ADA DATA TERHAPUS");
    }
    System.out.println(
        x: " "
    );

    if (!ada) return;

    System.out.print(s: "\nMasukkan ID yang ingin direstore: ");
    String id = in.next();

    for (int i = 0; i < n; i++) {
        if (!data[i].aktif &&
            data[i].id.equalsIgnoreCase(id)) {
            data[i].aktif = true;
            simpan();
            System.out.println(HIJAU +
                "Data berhasil direstore!" + RESET);
            return;
        }
    }

    System.out.println(MERAH +
        "Data tidak ditemukan!" + RESET);
}
}
```

Fitur restore menggunakan Sequential Search untuk mencari data nonaktif berdasarkan ID lalu mengaktifkannya kembali:

```
data[i].aktif = false;
```

Kompleksitas restore data:
[O(n)]

karena sistem memeriksa data satu per satu.

3.3 Implementasi Searching

3.3.1 Sequential Search

Searching Nama

```
// ===== SEARCH NAMA =====
static void sequentialSearchNama(String cari) {
    boolean ketemu = false;
    int no = 1;
    System.out.println("\n");
    System.out.printf(" | %10s | \n", ...args: "HASIL PENCARIAN NAMA");
    System.out.println(" | ");

    System.out.printf(" | %4s | %8s | %20s | %20s | %6s | %6s | %6s | %8s | %7s | %15s | \n",
        ...args: "No", "ID", "Nama", "Program Studi", "Tugas", "UTS", "UAS", "Akhir", "Grade", "Status");
    System.out.println(" | ");

    for (int i = 0; i < n; i++) {
        if (data[i].aktif &&
            data[i].nama.toLowerCase().contains(cari)) {
            ketemu = true;

            System.out.printf(" | %4d | %8s | %20s | %20s | %6.1f | %6.1f | %6.1f | %8.2f | %7s | %15s | \n",
                no++,
                data[i].id,
                data[i].nama,
                data[i].prodi,
                data[i].tugas,
                data[i].uts,
                data[i].uas,
                data[i].akhir,
                data[i].grade,
                data[i].status);
        }
    }

    if (!ketemu) {
        System.out.println(" | " + " ".repeat(count: 45)
            + "DATA TIDAK DITEMUKAN"
            + " ".repeat(count: 45) + " | ");
    }

    System.out.println(" | ");
}

static void cariNama() {
    System.out.print("Nama: ");
    String cari = in.nextline().toLowerCase();

    // urutkan dulu berdasarkan nama
    quickSortNama(low: 0, n - 1);

    // lakukan searching
    sequentialSearchNama(cari);
}
```

```
// ===== SEARCH PRODI =====
static void sequentialSearchProdi(String cari) {
    boolean ketemu = false;
    int no = 1;
    System.out.println(s: "no=");
    System.out.printf(format: "%10s%10s", "no", ""); //args: "HASIL PEMCARIAN PRODI");
    System.out.println(s: "");
    System.out.printf(format: "%10s%10s%10s%10s%10s%10s%10s%10s%10s%10s",
        "", args: "No", "ID", "Nama", "Program Studi", "Tugas", "UAS", "Mahir", "Grade", "Status");
    System.out.println(s: "");
    for (int i = 0; i < n; i++) {
        if (data[i].aktif && data[i].prodi.equalsIgnoreCase(cari)) {
            ketemu = true;
            System.out.printf(format: "%10s%10s%10s%10s%10s%10s%10s%10s%10s%10s",
                no,
                data[i].id,
                data[i].nama,
                data[i].prodi,
                data[i].tugas,
                data[i].uas,
                data[i].mahir,
                data[i].grade,
                data[i].status);
        }
        if (ketemu) {
            System.out.println(s: " " + " ".repeat(count: 45) + "DATA TIDAK DITEMUKAN" + " ".repeat(count: 45) + "");
        }
        System.out.println(s: "");
    }
}

static void cariProdi() {
    // array menyimpan prodi unik
    String[] daftarProdi = new String[100];
    int jumlahProdi = 0;

    // ambil semua prodi unik
    for (int i = 0; i < n; i++) {
        if (data[i].aktif) {
            boolean ada = false;
            for (int j = 0; j < jumlahProdi; j++) {
                if (daftarProdi[j].equalsIgnoreCase(data[i].prodi)) {
                    ada = true;
                    break;
                }
            }
            if (!ada) {
                daftarProdi[jumlahProdi++] = data[i].prodi;
            }
        }
    }

    // tampilkan daftar prodi
    System.out.println(s: "Pilih Prodi:");
    for (int i = 0; i < jumlahProdi; i++) {
        System.out.println((i + 1) + ". " + daftarProdi[i]);
    }

    System.out.println(s: "Pilih:");
    int pilih = InputAngka();
    teks: "Pilih: ", sio: i, jumlahProdi
};

// validasi
if (pilih < 1 || pilih > jumlahProdi) {
    System.out.println(s: "Pilihan tidak valid");
    return;
}

String cari = daftarProdi[pilih - 1];

// tampilkan nama data
quickSortNama(low: 0, n - 1);

// searching prodi
sequentialSearchProdi(cari);
}
```

- pencarian nama,
- pencarian program studi.

Kompleksitas:
[O(n)]

3.3.2 Binary Search

```
// ===== SEARCH ID =====
static int binarySearchID(String id) {
    int left = 0;
    int right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        int compare = data[mid].id.compareTo(id);
        // data ditemukan
        if (compare == 0) {
            return mid;
        }
        // cari ke kanan
        else if (compare < 0) {
            left = mid + 1;
        }
        // cari ke kiri
        else {
            right = mid - 1;
        }
    }
    // tidak ditemukan
    return -1;
}

static void cariID() {
    System.out.print(s: "ID: ");
    String id = in.next();

    // pastikan data terurut
    sortIDSilent();
    // cari dengan binary search
    int index = binarySearchID(id);
    System.out.println(x: "\n");
    System.out.printf(format: "%-106s | \n", ...args: "HASIL PENCARIAN ID");
    System.out.println(x: "|");
    System.out.printf(format: "%-4s|%-8s|%-20s|%-20s|%-6s|%-6s|%-8s|%-7s|%-15s| \n",
        ...args: "No", "ID", "Nama", "Program Studi", "Tugas", "UTS", "UAS", "Akhir", "Grade", "Status");
    System.out.println(x: "|");
    if (index != -1 && data[index].aktif) {
        System.out.printf(format: "%-4s|%-8s|%-20s|%-20s|%-6.1f|%-6.1f|%-6.1f|%-8.2f|%-7s|%-15s| \n",
            ...args: 1,
            data[index].id,
            data[index].nama,
            data[index].prodi,
            data[index].tugas,
            data[index].uts,
            data[index].uas,
            data[index].akhir,
            data[index].grade,
            data[index].status);
    } else {
        System.out.println("|" + " ".repeat(count: 45) + "DATA TIDAK DITEMUKAN" + " ".repeat(count: 45) + "|");
    }
    System.out.println(x: "|");
}
```

Binary Search digunakan pada pencarian ID mahasiswa.

Sebelum searching dilakukan, data terlebih dahulu diurutkan menggunakan *Quick Sort* berdasarkan ID. Setelah itu Binary Search membagi data menjadi dua bagian untuk mempercepat pencarian.

Kompleksitas:

$[O(\log n)]$

3.4 Implementasi Sorting

Program menggunakan algoritma *Quick Sort* untuk:

- sorting ID,

```
// ===== SORT ID =====
static void quickSortID(int low, int high) {
    if (low < high) {
        int pi = partitionID(low, high);
        quickSortID(low, pi - 1);
        quickSortID(pi + 1, high);
    }
}

static int partitionID(int low, int high) {
    String pivot = data[high].id;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (data[j].id.compareTo(pivot) < 0) {
            i++;
            Mahasiswa temp = data[i];
            data[i] = data[j];
            data[j] = temp;
        }
    }
    Mahasiswa temp = data[i + 1];
    data[i + 1] = data[high];
    data[high] = temp;
    return i + 1;
}

static void sortID() {
    quickSortID(low: 0, n - 1);
    tampil();
}
```

- sorting nama,

```
// ===== SORT NAMA =====
static void quickSortNama(int low, int high) {
    if (low < high) {
        int pi = partitionNama(low, high);
        quickSortNama(low, pi - 1);
        quickSortNama(pi + 1, high);
    }
}

static int partitionNama(int low, int high) {
    String pivot = data[high].nama.toLowerCase();
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (data[j].nama.toLowerCase().compareTo(pivot) < 0) {
            i++;
            Mahasiswa temp = data[i];
            data[i] = data[j];
            data[j] = temp;
        }
    }
    Mahasiswa temp = data[i + 1];
    data[i + 1] = data[high];
    data[high] = temp;
    return i + 1;
}

static void sortNama() {
    quickSortNama(low: 0, n - 1);
    tampil();
}
```

- sorting nilai akhir.

```
// ===== SORT NILAI =====
static void quickSortNilai(int low, int high) {
    if (low < high) {
        int pi = partition(low, high);
        quickSortNilai(low, pi - 1);
        quickSortNilai(pi + 1, high);
    }
}

static int partition(int low, int high) {
    double pivot = data[high].akhir;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (data[j].akhir > pivot) {
            i++;
            Mahasiswa temp = data[i];
            data[i] = data[j];
            data[j] = temp;
        }
    }
    Mahasiswa temp = data[i + 1];
    data[i + 1] = data[high];
    data[high] = temp;

    return i + 1;
}

static void sortNilai() {
    quickSortNilai(low: 0, n - 1);
    tampil();
}
```

Quick Sort menggunakan konsep pivot dan partition untuk membagi data menjadi beberapa bagian lalu diurutkan secara rekursif.

Kompleksitas rata-rata:
[$O(n \log n)$]

Quick Sort dipilih karena lebih cepat dibanding *Bubble Sort* ketika jumlah data banyak.

3.5 Implementasi Statistik

```
// ===== STATISTIK =====
static void statistik() {
    int total = 0, lulus = 0, tidak = 0;
    int A=0,B=0,C=0,D=0,E=0;
    double sum = 0;
    for (int i = 0; i < n; i++) {
        if (data[i].aktif) {
            total++;
            sum += data[i].akhir;
            if (data[i].status.equals(anObject: "LULUS")) lulus++;
            else tidak++;
            switch (data[i].grade) {
                case "A" -> A++;
                case "B" -> B++;
                case "C" -> C++;
                case "D" -> D++;
                case "E" -> E++;
            }
        }
    }
}
```

Fitur statistik digunakan untuk menampilkan:

- total mahasiswa,
- jumlah lulus,
- jumlah tidak lulus,
- rata-rata nilai,
- persentase kelulusan,
- distribusi grade A–E.

Program menggunakan perulangan dan percabangan untuk menghitung seluruh data statistik mahasiswa secara otomatis.

3.6 Implementasi File Handling

Program menggunakan file handling untuk menyimpan dan membaca data mahasiswa melalui file `data.txt`.

3.6.1 Simpan Data

```
static void sortIDSilent() {
    quickSortID(low: 0, n - 1);
}
static void simpan() {
    sortIDSilent();
    try {
        PrintWriter w =
            new PrintWriter(fileName: "data.txt");
        for (int i = 0; i < n; i++) {
            w.println(
                data[i].id + "," +
                data[i].nama + "," +
                data[i].prodi + "," +
                data[i].tugas + "," +
                data[i].uts + "," +
                data[i].uas + "," +
                data[i].aktif
            );
        }
        w.close();
        System.out.println(
            x: "Auto save berhasil!");
    } catch (Exception e) {
        System.out.println(
            x: "Gagal auto save!");
    }
}
```

Data disimpan menggunakan:

```
PrintWriter w =
    new PrintWriter(fileName: "data.txt");
```

Program melakukan auto save setiap ada perubahan data.

3.6.2 Load Data

```
static void loadData() {
    try {
        File file = new File(pathname: "data.txt");
        if (!file.exists()) return;
        Scanner baca = new Scanner(file);
        while (baca.hasNextLine()) {
            String[] d = baca.nextLine().split(regex: ",");
            data[n] = new Mahasiswa(
                d[0],
                d[1],
                d[2],
                Double.parseDouble(d[3]),
                Double.parseDouble(d[4]),
                Double.parseDouble(d[5])
            );
            data[n].aktif =
                Boolean.parseBoolean(d[6]);
            n++;
        }
        baca.close();
        System.out.println(x: "Data berhasil dimuat!");
    } catch (Exception e) {
        System.out.println(x: "Gagal load data!");
    }
}
```

Data dibaca kembali menggunakan:

```
Scanner baca = new Scanner(file);
```

Sehingga data tetap tersedia ketika program dijalankan ulang.

3.7 Implementasi Login dan Admin

```
// ===== LOGIN =====
static void login() {

    while (true) {

        System.out.println();

        System.out.println(CYAN + BOLD);
        System.out.println(x: "=====");
        System.out.printf (format: "%-28s \n", ...args: "LOGIN SISTEM");
        System.out.println("===== " + RESET);

        // INPUT USERNAME
        System.out.print(KUNING + "Username / ID : " + RESET);
        String username = in.nextLine();

        // INPUT PASSWORD
        System.out.print(KUNING + "Password      : " + RESET);
        String password = in.nextLine();

    }

}
```

Program memiliki dua role:

```
// ===== LOGIN ADMIN =====
if (username.equals(anObject: "admin") &&
    password.equals(anObject: "123")) {

    role = "admin";

    System.out.println(HIJAU +
        "\nLogin Admin Berhasil!"
        + RESET);

    break;
}

// ===== LOGIN USER =====
else {

    boolean ketemu = false;

    for (int i = 0; i < n; i++) {

        if (data[i].aktif &&
            data[i].id.equalsIgnoreCase(username) &&
            password.equals(anObject: "123")) {

            role = "user";
            loginID = data[i].id;
            ketemu = true;

        }

    }

}
```

1. Admin
2. User

3.7.1 Admin

```
// ----- MENU ADMIN -----  
if (role.equals(anObject: "admin")) {  
  
    switch (p) {  
        case 1 -> tambah();  
        case 2 -> tampil();  
        case 3 -> edit();  
        case 4 -> hapus();  
        case 5 -> cariNama();  
        case 6 -> cariID();  
        case 7 -> cariProdi();  
        case 8 -> sortID();  
        case 9 -> sortNama();  
        case 10 -> sortNilai();  
        case 11 -> statistik();  
        case 12 -> restore();  
        case 13 -> {  
            logout();  
        }  
        case 0 -> {  
            System.out.println(HIJAU +  
                "\nTerima kasih telah menggunakan sistem"  
                + RESET);  
        }  
        default -> {  
            System.out.println(MERAH +  
                "Menu tidak tersedia!" + RESET);  
        }  
    }  
}  
  
    } while (p != 0);  
}
```

Admin dapat:

- tambah data,
- edit data,
- hapus data,
- searching,
- sorting,
- Statistik,
- restore data.

3.7.2 User

```
// ----- MENU | USER -----
else {
    for (int i = 0; i < n; i++) {
        if (data[i].aktif &&
            data[i].id.equalsIgnoreCase(loginID)) {
            System.out.println(
                x: "\n");

            System.out.printf(
                format: "%-44s \n",
                ...args: "DATA MAHASISMA");

            System.out.println(
                x: " ");

            System.out.printf(
                format: "%-16s | %-25s \n",
                ...args: "ID",
                data[i].id);

            System.out.printf(
                format: "%-16s | %-25s \n",
                ...args: "Nama",
                data[i].nama);

            System.out.printf(
                format: "%-16s | %-25s \n",
                ...args: "Prodi",
                data[i].prodi);

            System.out.printf(
                format: "%-16s | %-25.1f \n",
                ...args: "Tugas",
                data[i].tugas);

            System.out.printf(
                format: "%-16s | %-25.1f \n",
                ...args: "UTS",
                data[i].uts);

            System.out.printf(
                format: "%-16s | %-25.1f \n",
                ...args: "UAS",
                data[i].uas);

            System.out.printf(
                format: "%-16s | %-25.2f \n",
                ...args: "Nilai Akhir",
                data[i].akhir);

            System.out.printf(
                format: "%-16s | %-25s \n",
                ...args: "Grade",
                data[i].grade);

            System.out.printf(
                format: "%-16s | %-25s \n",
                ...args: "Status",
                data[i].status);

            System.out.println(
                x: " ");
        }
    }
}
```

User hanya dapat melihat data miliknya sendiri berdasarkan ID login.

3.8 Implementasi Interface

Program menggunakan tampilan console interaktif dengan:

- tabel ASCII,

```
format: "%-4s%-8s%-20s%-20s%-6s%-6s%-6s%-8s%-7s%-15s |\n",
```

- warna terminal ANSI color,

```
// ===== WARNA =====
static final String RESET = "\u001B[0m";
static final String MERAH = "\u001B[31m";
static final String HIJAU = "\u001B[32m";
static final String KUNING = "\u001B[33m";
static final String BIRU = "\u001B[34m";
static final String UNGU = "\u001B[35m";
static final String CYAN = "\u001B[36m";
static final String BOLD = "\u001B[1m";
```

- loading animation,

```
// ===== ANIMASI =====
static void loading(String teks) {
    System.out.print(CYAN + teks);
    try {
        for (int i = 0; i < 5; i++) {
            Thread.sleep(200);
            System.out.print(s: ".");
        }
    } catch (Exception e) {}
    System.out.println(RESET);
}

static void progressBar() {
    System.out.print(KUNING + "Loading: ");
    try {
        for (int i = 0; i <= 20; i++) {
            System.out.print(s: "█");
            Thread.sleep(20);
        }
    } catch (Exception e) {}
    System.out.println(" 100%" + RESET);
}
```

- progress bar.

```
for (int i = 0; i <= 20; i++) {
    System.out.print(s: "█");
    Thread.sleep(20);
}
```

- Logout Session

```
system.out.println(KUNING +
    "\nSession berakhir..." + RESET);
// animasi loading
loading(teks: "Mengalihkan ke halaman login");
// reset pola
```

Tujuan implementasi interface ini adalah agar program lebih rapi, menarik, dan mudah digunakan pengguna.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Pendahuluan

Analisis kompleksitas merupakan proses untuk mengukur tingkat efisiensi suatu algoritma berdasarkan kebutuhan waktu eksekusi (*time complexity*) dan penggunaan memori (*space complexity*). Analisis ini sangat penting dalam pengembangan perangkat lunak karena dapat menentukan performa sistem ketika menangani data dalam jumlah besar.

Pada penelitian ini, sistem yang dianalisis adalah program *Sistem Manajemen Nilai Mahasiswa* berbasis bahasa pemrograman *Java*. Program menggunakan struktur data array bertipe objek *Mahasiswa[]* dengan kapasitas maksimum 200 data. Sistem memiliki beberapa fitur utama seperti login, tambah data, edit data, hapus data, pencarian data, pengurutan data, statistik nilai, serta penyimpanan data ke file.

Dalam implementasinya, program menerapkan beberapa algoritma penting, yaitu:

- *Sequential Search*
- *Binary Search*
- *Quick Sort*
- Perulangan (*looping*)
- Operasi file (*file handling*)
- Validasi input

Analisis kompleksitas dilakukan untuk mengetahui tingkat efisiensi dari setiap proses sehingga dapat dievaluasi apakah algoritma yang digunakan sudah optimal atau masih memiliki keterbatasan performa.

4.2 Analisis Kompleksitas Waktu

Kompleksitas waktu (*time complexity*) menggambarkan jumlah operasi yang dilakukan algoritma terhadap ukuran data masukan (n). Semakin besar nilai kompleksitas, maka semakin lama waktu eksekusi program ketika jumlah data bertambah.

Pada program ini terdapat beberapa proses utama yang dianalisis, yaitu:

4.2.1 Proses Login

Proses login dilakukan dengan mencocokkan username dan password terhadap data mahasiswa yang tersimpan pada array. Sistem melakukan pengecekan satu per satu menggunakan perulangan.

```
for (int i = 0; i < n; i++)
```

Karena proses pencarian dilakukan secara linear, maka kompleksitas waktunya adalah:

$$O(n)$$

Artinya waktu eksekusi bertambah seiring bertambahnya jumlah data mahasiswa.

4.2.2 Proses Tambah Data

Pada proses penambahan data, sistem melakukan:

1. Pengecekan ID agar tidak duplikat
2. Input data mahasiswa
3. Penyimpanan data ke array
4. Penyimpanan ke file

Pengecekan ID menggunakan metode cekID() yang melakukan pencarian linear terhadap seluruh data.

Kompleksitas waktunya:

$$O(n)$$

Sedangkan proses penyimpanan array ke file juga membaca seluruh data sehingga:

$$O(n)$$

Total kompleksitas proses tambah data menjadi:

$$O(n)$$

4.2.3 Proses Edit Data

Proses edit menggunakan fungsi `cariIndexID(id)`. Fungsi tersebut melakukan pencarian linear terhadap array mahasiswa.

Kompleksitas waktunya:

$$O(n)$$

Jika data ditemukan, perubahan data dilakukan secara langsung dengan kompleksitas:

$$O(1)$$

Namun karena pencarian mendominasi proses, maka total kompleksitas tetap:

$$O(n)$$

4.2.4 Proses Hapus Data

Proses penghapusan data menggunakan metode *soft delete*, yaitu mengubah atribut aktif menjadi false. Sebelum mengubah status, sistem mencari data berdasarkan ID menggunakan pencarian linear.

Kompleksitas waktunya:

$$O(n)$$

4.2.5 Proses Pencarian Nama

Pencarian nama menggunakan algoritma *Sequential Search*.

```
if (data[i].nama.toLowerCase().contains(cari))
```

Sistem memeriksa seluruh data satu per satu sehingga kompleksitas waktunya:

$$O(n)$$

Walaupun sebelumnya dilakukan pengurutan menggunakan *Quick Sort*, proses pencarian tetap linear karena sistem mencari kemungkinan banyak hasil.

4.2.6 Proses Pencarian ID

Pencarian ID menggunakan algoritma *Binary Search*.

```
while (left <= right)
```

Binary Search membagi data menjadi dua bagian setiap iterasi sehingga kompleksitas waktunya:

$$O(\log n)$$

Namun sebelum pencarian dilakukan, sistem melakukan pengurutan data menggunakan *Quick Sort*.

Kompleksitas *Quick Sort* rata-rata:

$$O(n \log n)$$

Sehingga total proses menjadi:

$$O(n \log n)$$

4.2.7 Proses Pengurutan Data

Program menggunakan algoritma *Quick Sort* pada:

- *Sort* ID
- *Sort* Nama
- *Sort* Nilai

Quick Sort bekerja dengan metode *divide and conquer* menggunakan pivot.

Kompleksitas rata-rata:

$$O(n \log n)$$

Kompleksitas terburuk:

$$O(n^2)$$

Kondisi terburuk terjadi ketika pivot selalu menghasilkan pembagian data yang tidak seimbang.

4.2.8 Proses Statistik

Fitur statistik melakukan iterasi terhadap seluruh data mahasiswa untuk menghitung:

- Total mahasiswa
- Jumlah lulus

- Distribusi *grade*
- Nilai rata-rata

Karena seluruh data dibaca satu kali, maka kompleksitas waktunya:

$$O(n)$$

4.2.9 Proses Simpan Data

Proses penyimpanan data dilakukan menggunakan `PrintWriter`.

Sistem menulis seluruh isi array ke file `data.txt`.

Kompleksitas waktunya:

$$O(n)$$

Karena seluruh data harus diproses sebelum file ditutup.

4.3 Analisis Kompleksitas Ruang

Kompleksitas ruang (*space complexity*) menunjukkan jumlah memori tambahan yang digunakan algoritma selama proses eksekusi.

4.3.1 Struktur Data Array

Program menggunakan array:

```
Mahasiswa[] data = new Mahasiswa[200];
```

Penggunaan memori utama bersifat tetap sebesar kapasitas array.

Kompleksitas ruang:

$$O(n)$$

4.3.2 Quick Sort

Quick Sort menggunakan rekursi sehingga membutuhkan memori tambahan pada *call stack*.

Kompleksitas ruang rata-rata:

$$O(\log n)$$

Kompleksitas terburuk:

$$O(n)$$

4.3.3 Sequential Search dan Binary Search

Kedua algoritma pencarian tidak menggunakan struktur data tambahan yang besar.

Kompleksitas ruang:

$$O(1)$$

4.3.4 Penyimpanan File

Saat proses simpan dan load data, program menggunakan variabel sementara untuk membaca isi file.

Kompleksitas ruang:

$$O(1)$$

Karena data langsung diproses tanpa menyimpan salinan besar tambahan.

4.4 Tabel Analisis Kompleksitas

No	Fungsi/Proses	Deskripsi	Time Complexity	Space Complexity	Penjelasan
1	Login	Validasi <i>username</i> dan password	$O(n)$	$O(1)$	Mengecek data satu per satu
2	Tambah Data	Menambahkan data mahasiswa	$O(n)$	$O(1)$	Ada pengecekan ID dan penyimpanan
3	Edit Data	Mengubah data mahasiswa	$O(n)$	$O(1)$	Menggunakan pencarian linear
4	Hapus Data	Menonaktifkan data	$O(n)$	$O(1)$	Pencarian ID dilakukan linear
5	Restore Data	Mengaktifkan kembali data	$O(n)$	$O(1)$	Membaca seluruh data terhapus
6	Cari Nama	<i>Sequential Search</i>	$O(n)$	$O(1)$	Memeriksa seluruh array
7	Cari ID	<i>Binary Search</i>	$O(\log n)$	$O(1)$	Data harus terurut
8	Cari Prodi	<i>Sequential Search</i>	$O(n)$	$O(1)$	Membandingkan seluruh prodi

9	Sort ID	Quick Sort	$O(\log n)$	$O(\log n)$	Menggunakan rekursi
10	Sort Nama	Quick Sort	$O(\log n)$	$O(\log n)$	Menggunakan pivot
11	Sort Nilai	Quick Sort	$O(\log n)$	$O(\log n)$	Pengurutan descending
12	Statistik	Perhitungan nilai	$O(n)$	$O(1)$	Membaca semua data
13	Simpan Data	Menulis ke file	$O(n)$	$O(1)$	Semua data ditulis
14	Load Data	Membaca file	$O(n)$	$O(1)$	Data dibaca satu per satu

Berdasarkan hasil analisis kompleksitas, dapat diketahui bahwa performa sistem sangat dipengaruhi oleh penggunaan array sebagai struktur data utama. Array memiliki kelebihan dalam akses data yang cepat menggunakan indeks, namun memiliki kelemahan pada proses pencarian dan manipulasi data karena harus dilakukan secara linear.

Algoritma *Sequential Search* digunakan pada pencarian nama dan program studi. Algoritma ini cukup sederhana dan mudah diimplementasikan, namun kurang efisien ketika jumlah data semakin besar karena harus memeriksa seluruh elemen array satu per satu.

Sementara itu, pencarian berdasarkan ID menggunakan *Binary Search* yang jauh lebih efisien dibanding pencarian linear. *Binary Search* mampu mengurangi ruang pencarian menjadi setengah pada setiap iterasi sehingga kompleksitas waktunya hanya:

$$O(\log n)$$

Namun Binary Search memiliki syarat bahwa data harus dalam kondisi terurut. Oleh karena itu sistem melakukan proses sorting terlebih dahulu menggunakan Quick Sort.

Penggunaan Quick Sort pada program merupakan pilihan yang tepat karena algoritma ini memiliki performa rata-rata yang sangat baik, yaitu:

$$O(n \log n)$$

Selain cepat, Quick Sort juga relatif hemat memori karena proses pengurutan dilakukan secara *in-place*.

Walaupun demikian, Quick Sort memiliki kelemahan pada kondisi terburuk ketika pivot yang dipilih tidak optimal. Pada kondisi tersebut kompleksitas dapat meningkat menjadi:

$$O(n^2)$$

Dalam sistem ini, pemilihan pivot menggunakan elemen terakhir array sehingga kemungkinan kasus terburuk tetap dapat terjadi apabila data sudah hampir terurut.

Dari sisi memori, program tergolong efisien karena sebagian besar proses hanya menggunakan variabel sederhana tanpa struktur data tambahan yang besar. Penggunaan array statis juga membuat penggunaan memori lebih stabil.

Namun terdapat beberapa keterbatasan pada sistem, antara lain:

1. Kapasitas data maksimal hanya 200 mahasiswa.
2. Proses pencarian masih banyak menggunakan metode linear.
3. Penyimpanan file masih berbasis teks biasa (*text file*).
4. Belum menggunakan database sehingga skalabilitas terbatas.

Apabila sistem dikembangkan lebih lanjut, beberapa optimasi yang dapat dilakukan adalah:

- Menggunakan *ArrayList*
- Menggunakan *HashMap* untuk pencarian ID
- Menggunakan *database* MySQL
- Mengimplementasikan algoritma sorting yang lebih stabil
- Menggunakan *indexing* data

Dengan optimasi tersebut, performa sistem dapat meningkat secara signifikan terutama ketika jumlah data semakin besar.

BAB V

PENGUJIAN

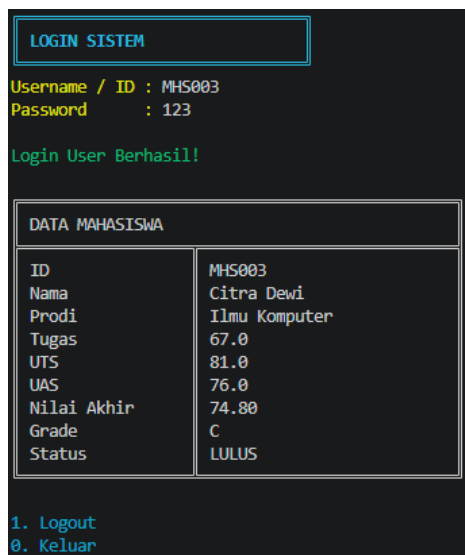
5.1 Skenario Pengujian

Pengujian dilakukan terhadap seluruh fitur sistem secara manual menggunakan berbagai skenario input, termasuk skenario normal (valid) dan skenario tepi (*edge case*). Berikut skenario yang dijalankan serta output screenshot.

5.1.1 Skenario 1: Login akun user dan admin

Awal program distart akan memberikan tampilan "login sistem" disini, user diharuskan untuk menginput *ID/Username* dan *password* mereka. Hasil output akan memberikan tabel data penilaian untuk mahasiswa tersebut.

Teruntuk login ke Akun Admin, perlu menginput username "admin" dan password yang sama. Hasil yang diberikan berupa tabel dengan fitur untuk memanipulasi data.



Gambar 5.1.1.1 Output tabel mahasiswa



Gambar 5.1.1.2 Tabel interface admin

5.1.2 Skenario 2: Tampilkan data

Dengan akun admin dan menginput nilai 2 diterminal akan menghasilkan tabel nilai seluruh mahasiswa.

Pilih menu: 2

SISTEM NILAI MAHASISWA DATA AKTIF									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
1	MHS002	Budi Santoso	Sistem Informasi	88.0	73.0	95.0	86.30	A	LULUS
2	MHS003	Citra Dewi	Ilmu Komputer	67.0	81.0	76.0	74.80	C	LULUS
3	MHS004	Dian Purnama	Teknik Informatika	90.0	64.0	71.0	74.60	C	LULUS
4	MHS005	Eko Prasetyo	Sistem Informasi	74.0	89.0	68.0	76.10	B	LULUS
5	MHS006	Fitri Handayani	Ilmu Komputer	96.0	87.0	93.0	92.10	A	LULUS
6	MHS007	Gilang Ramadhan	Teknik Informatika	72.0	84.0	79.0	78.40	B	LULUS
7	MHS008	Hana Safitri	Sistem Informasi	65.0	77.0	83.0	75.80	B	LULUS
8	MHS009	Irfan Maulana	Ilmu Komputer	52.0	57.0	59.0	56.30	D	TIDAK LULUS
9	MHS010	Joko Widodo	Teknik Informatika	91.0	88.0	80.0	85.70	A	LULUS
10	MHS011	Kartika Sari	Sistem Informasi	63.0	74.0	67.0	67.90	C	LULUS
11	MHS012	Luthfi Hakim	Ilmu Komputer	86.0	79.0	72.0	78.30	B	LULUS
12	MHS013	Maya Anggraini	Teknik Informatika	98.0	94.0	97.0	96.40	A	LULUS
13	MHS014	Nanda Permata	Sistem Informasi	70.0	82.0	76.0	76.00	B	LULUS
14	MHS015	Oka Mahendra	Ilmu Komputer	84.0	68.0	91.0	82.00	B	LULUS
15	MHS016	Putri Rahayu	Teknik Informatika	73.0	61.0	88.0	75.40	B	LULUS
16	MHS017	Qodir Abdillah	Sistem Informasi	69.0	75.0	90.0	79.20	B	LULUS
17	MHS018	Rina Marlina	Ilmu Komputer	95.0	97.0	92.0	94.40	A	LULUS
18	MHS019	Sari Indah	Teknik Informatika	89.0	78.0	66.0	76.50	B	LULUS
19	MHS020	Teguh Prasetya	Sistem Informasi	81.0	72.0	87.0	80.70	B	LULUS
20	MHS021	Umi Kalsum	Ilmu Komputer	94.0	85.0	79.0	85.30	A	LULUS
21	MHS022	Vina Andriani	Teknik Informatika	76.0	63.0	84.0	75.30	B	LULUS
22	MHS023	Wahyu Hidayat	Sistem Informasi	92.0	86.0	74.0	83.00	B	LULUS
23	MHS024	Xena Oktavia	Ilmu Komputer	87.0	71.0	93.0	84.60	B	LULUS
24	MHS025	Yusuf Hidayah	Teknik Informatika	62.0	83.0	69.0	71.10	C	LULUS
25	MHS026	Zahra Nabila	Sistem Informasi	96.0	89.0	91.0	91.90	A	LULUS
26	MHS027	Andi Kurniawan	Ilmu Komputer	71.0	66.0	82.0	73.90	C	LULUS
27	MHS028	Bella Safira	Teknik Informatika	68.0	74.0	77.0	73.40	C	LULUS
28	MHS029	Chandra Gumilar	Sistem Informasi	79.0	70.0	94.0	82.30	B	LULUS
29	MHS030	Dewi Nurhaliza	Ilmu Komputer	64.0	78.0	99.0	82.20	B	LULUS
30	MHS031	Bryan	Sistem Informasi	89.0	99.0	100.0	96.40	A	LULUS
31	MHS032	Reyhan	Teknik Informatika	100.0	89.0	89.0	92.30	A	LULUS
32	MHS033	Jacob Mamo	Sistem Informasi	10.0	38.0	5.0	16.40	E	TIDAK LULUS

Gambar 5.1.2 Tampilan tabel data nilai mahasiswa

5.1.3 Skenario 3: Tambah data mahasiswa

Menginput angka 1 diterminal untuk menambahkan data mahasiswa baru, program akan memberikan output yang berupa data yang relevan untuk mahasiswa tersebut. Data yang diinput tersebut berupa ID, nama, prodi, dan nilai-nilai. Setelah itu, data akan berhasil ditambahkan dan *auto-save* data baru tersebut ke dalam data.txt.

```
Masukkan angka sesuai menu yang dipilih
Pilih menu: 1
ID terakhir yang digunakan: MHS034
Masukkan ID: MHS035
Nama: Tom Halland
Prodi: Teknik Informatika
Tugas: 78
UTS: 69
UAS: 89
Data berhasil ditambahkan!
Auto save berhasil!
```

Gambar 5.1.3 Tambah data mahasiswa

5.1.4 Skenario 4: Menambah data dengan ID duplikat

Saat melakukan penambahan data baru, namun saat input ID yang tidak sengaja ternyata duplikat, program memberikan peringatan dan menjalankan ulang input data baru.

```
Pilih menu: 1
ID terakhir yang digunakan: MHS035
Masukkan ID: MHS003
ID sudah ada! Gunakan ID lain.
Masukkan ID: 
```

Gambar 5.1.4 Peringatan duplikat ID

5.1.5 Skenario 5: Edit data

Fitur edit dapat digunakan dengan input angka 3 dimenu, program akan mengoutputkan data yang berelevan dari mahasiswa tersebut dan memberikan pilihan untuk benar atau tidaknya megubah suatu data.

```
Masukkan angka sesuai menu yang dipilih
Pilih menu: 3
ID: MHS033
Ubah Nama? (ya/tidak): ya
Nama baru: Jacob Mamoa
Ubah Prodi? (ya/tidak): ya
Prodi baru: Sistem Informasi
Ubah Nilai? (ya/tidak): ya
Tugas: 78
UTS: 88
UAS: 92
Auto save berhasil!
```

Gambar 5.1.5.1 Output edit data

31	MHS032	Reyhan	Teknik Informatika	100.0	89.0	89.0	92.30	A	LULUS
32	MHS033	Jacob Mamoa	Sistem Informasi	10.0	38.0	5.0	16.40	E	TIDAK LULUS
33	MHS034	Vladimir Putin	Sistem Informasi	99.0	99.0	99.0	99.00	A	LULUS

Gambar 5.1.5.2 Data sebelum diedit

31	MHS032	Reyhan	Teknik Informatika	100.0	89.0	89.0	92.30	A	LULUS
32	MHS033	Jacob Mamoa	Sistem Informasi	78.0	88.0	92.0	86.60	A	LULUS
33	MHS034	Vladimir Putin	Sistem Informasi	99.0	99.0	99.0	99.00	A	LULUS

Gambar 5.1.5.3 Data setelah diedit

5.1.6 Skenario 6: Hapus dan restore data mahasiswa

Fitur hapus diprogram menggunakan *soft delete* dimana tidak benar-benar terhapus dari *array*, melainkan field aktif diset menjadi

false. Efeknya, data tidak akan muncul pada terampilan tabel, dll, namun tetap tersimpan di data.txt.

```
Masukkan angka sesuai menu yang dipilih
Pilih menu: 4
ID: MHS035
Auto save berhasil!
Data dihapus!
```

Gambar 5.1.6.1 Hapus data

30	MHS031	Bryan
31	MHS032	Reyhan
32	MHS033	Jacob Mamo
33	MHS034	Vladimir Putin
34	MHS036	Josh Buck

Gambar 5.1.6.2 Hasil tabel data dihapus

```
33 MHS034,Vladimir Putin,Sistem Informasi,99.0,99.0,99.0,true
34 MHS035,Tom Halland,Teknik Informatika,78.0,69.0,89.0,false
35 MHS036,Josh Buck,Teknik Informatika,56.0,78.0,67.0,true
```

Gambar 5.1.6.3 File data.txt setelah data dihapus

Menggunakan fitur *restore*, program akan meng-outputkan tabel yang berupa data-data yang mungkin telah dihapuskan. Inputkan ID yang ingin dipulihkan kembali dan data tersebut akan berhasil direstore. *Field* data yang diset menjadi *false* akan menjadi *true* kembali. Sehingga, data yang dihapus akan kembali aktif dan dapat ditampilkan ke tabel.

```
Pilih menu: 12

DATA MAHASISWA TERHAPUS


| No | ID     | Nama        | Program Studi      |
|----|--------|-------------|--------------------|
| 1  | MHS035 | Tom Halland | Teknik Informatika |



Masukkan ID yang ingin direstore: MHS035
Auto save berhasil!
Data berhasil direstore!
```

Gambar 5.1.6.4 Tabel data terhapus

```
33 MHS034,Vladimir Putin,Sistem Informasi,99.0,99.0,99.0,true
34 MHS035,Tom Halland,Teknik Informatika,78.0,69.0,89.0,true
35 MHS036,Josh Buck,Teknik Informatika,56.0,78.0,67.0,true
```

Gambar 5.1.6.5 data.txt setelah restore

5.1.7 Skenario 7: Pencarian nama mahasiswa

Fitur pencarian nama menggunakan pencocokan parsial (*contains*) sehingga tidak perlu mengetikkan nama lengkap, cukup sebagian saja. Program akan meng-outputkan tabel yang berisi list nama yang memiliki bagian kata tersebut.

Masukkan angka sesuai menu yang dipilih
Pilih menu: 5
Nama: ka

HASIL PENCARIAN NAMA									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
1	MHS011	Kartika Sari	Sistem Informasi	63.0	74.0	67.0	67.90	C	LULUS
2	MHS015	Oka Mahendra	Ilmu Komputer	84.0	68.0	91.0	82.00	B	LULUS
3	MHS021	Umi Kalsum	Ilmu Komputer	94.0	85.0	79.0	85.30	A	LULUS

Gambar 5.1.7.1 Tabel Search By Name

Bila mencari nama dengan mengetik nama lengkap apabila nama tersebut ada dalam data, maka pencarian tetap akan berhasil. Namun sebaliknya terjadi ketika data tidak terdapat nama tersebut, tabel akan meng-outputkan tabel bahwa nama tersebut tidak ditemukan/ada.

Pilih menu: 5
Nama: Stellarium

HASIL PENCARIAN NAMA									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
DATA TIDAK DITEMUKAN									

Gambar 5.1.7.2 Tabel Search Nama tidak ditemukan

5.1.8 Skenario 8: Pencarian dengan ID mahasiswa

Berbeda dengan *Search By ID*, pencarian ID menggunakan pencocokan tepat (*exact match*), sehingga ID yang harus dimasukkan harus sesuai dan persis.

Pilih menu: 6
ID: MHS007

HASIL PENCARIAN ID									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
1	MHS007	Gilang Ramadhan	Teknik Informatika	72.0	84.0	79.0	78.40	B	LULUS

Gambar 5.1.8.1 tabel search by id

Bila memasukkan ID yang tidak terdaftar dalam data ataupun salah input ID, maka program akan memberika tabel bahwa data tersebut tidak ditemukan/ada.

Pilih menu: 6
ID: MHS040

HASIL Pencarian ID									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
DATA TIDAK DITEMUKAN									

Gambar 5.1.8.2 Tabel Search ID tidak ditemukan

5.1.9 Skenario 9: Pencarian dengan Prodi mahasiswa

Fitur pencarian prodi memberikan pengguna pilihan untuk mencari data dari prodi yang tersedia dalam data. Program akan meng-outputkan tabel data mahasiswa yang memiliki prodi yang sesuai dengan pilihan *user*.

Masukkan angka sesuai menu yang dipilih
Pilih menu: 7
Pilih Prodi:
1. Sistem Informasi
2. Ilmu Komputer
3. Teknik Informatika
Pilihan: Pilihan: 1

HASIL Pencarian PRODI									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
1	MHS031	Bryan	Sistem Informasi	89.0	99.0	100.0	96.40	A	LULUS
2	MHS002	Budi Santoso	Sistem Informasi	88.0	73.0	95.0	86.30	A	LULUS
3	MHS029	Chandra Gumilar	Sistem Informasi	79.0	70.0	94.0	82.30	B	LULUS
4	MHS005	Eko Prasetyo	Sistem Informasi	74.0	89.0	68.0	76.10	B	LULUS
5	MHS008	Hana Safitri	Sistem Informasi	65.0	77.0	83.0	75.80	B	LULUS
6	MHS033	Jacob Mamoa	Sistem Informasi	78.0	88.0	92.0	86.60	A	LULUS
7	MHS011	Kartika Sari	Sistem Informasi	63.0	74.0	67.0	67.90	C	LULUS
8	MHS014	Nanda Permata	Sistem Informasi	70.0	82.0	76.0	76.00	B	LULUS
9	MHS017	Qodir Abdillah	Sistem Informasi	69.0	75.0	90.0	79.20	B	LULUS
10	MHS020	Teguh Prasetya	Sistem Informasi	81.0	72.0	87.0	80.70	B	LULUS
11	MHS034	Vladimir Putin	Sistem Informasi	99.0	99.0	99.0	99.00	A	LULUS
12	MHS023	Wahyu Hidayat	Sistem Informasi	92.0	86.0	74.0	83.00	B	LULUS
13	MHS026	Zahra Nabila	Sistem Informasi	96.0	89.0	91.0	91.90	A	LULUS

Gambar 5.1.9.1 Tabel Search by Prodi

Bila sebaliknya *user* salah input dipilihan, sistem akan memberikan peringatan dan akan mengulangi input dalam tahap pilihan.

Masukkan angka sesuai menu yang dipilih
Pilih menu: 7
Pilih Prodi:
1. Ilmu Komputer
2. Teknik Informatika
3. Sistem Informasi
Pilihan: Pilihan: 9
Input harus 1 - 3!
Pilihan: a
Input harus angka!
Pilihan: %
Input harus angka!
Pilihan: []

Gambar 5.1.9.2 Peringatan input data

5.1.10 Skenario 10: *Sorting By ID*

Fitur *sorting by id* memberikan tabel yang berisi urutan ID mahasiswa secara *ascending* (meningkat).

Masukkan angka sesuai menu yang dipilih
Pilih menu: 8

SISTEM NILAI MAHASISWA DATA AKTIF									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
1	MHS002	Budi Santoso	Sistem Informasi	88.0	73.0	95.0	86.30	A	LULUS
2	MHS003	Citra Dewi	Ilmu Komputer	67.0	81.0	76.0	74.80	C	LULUS
3	MHS004	Dian Purnama	Teknik Informatika	90.0	64.0	71.0	74.60	C	LULUS
4	MHS005	Eko Prasetyo	Sistem Informasi	74.0	89.0	68.0	76.10	B	LULUS
5	MHS006	Fitri Handayani	Ilmu Komputer	96.0	87.0	93.0	92.10	A	LULUS
6	MHS007	Gilang Ramadhan	Teknik Informatika	72.0	84.0	79.0	78.40	B	LULUS
7	MHS008	Hana Safitri	Sistem Informasi	65.0	77.0	83.0	75.80	B	LULUS
8	MHS009	Irfan Maulana	Ilmu Komputer	52.0	57.0	59.0	56.30	D	TIDAK LULUS
9	MHS010	Joko Widodo	Teknik Informatika	91.0	88.0	80.0	85.70	A	LULUS
10	MHS011	Kartika Sari	Sistem Informasi	63.0	74.0	67.0	67.90	C	LULUS
11	MHS012	Luthfi Hakim	Ilmu Komputer	86.0	79.0	72.0	78.30	B	LULUS
12	MHS013	Maya Anggraini	Teknik Informatika	98.0	94.0	97.0	96.40	A	LULUS
13	MHS014	Nanda Permata	Sistem Informasi	70.0	82.0	76.0	76.00	B	LULUS
14	MHS015	Oka Mahendra	Ilmu Komputer	84.0	68.0	91.0	82.00	B	LULUS
15	MHS016	Putri Rahayu	Teknik Informatika	73.0	61.0	88.0	75.40	B	LULUS
16	MHS017	Qodir Abdillah	Sistem Informasi	69.0	75.0	90.0	79.20	B	LULUS
17	MHS018	Rina Marlina	Ilmu Komputer	95.0	97.0	92.0	94.40	A	LULUS
18	MHS019	Sari Indah	Teknik Informatika	89.0	78.0	66.0	76.50	B	LULUS
19	MHS020	Teguh Prasetya	Sistem Informasi	81.0	72.0	87.0	80.70	B	LULUS
20	MHS021	Umi Kalsum	Ilmu Komputer	94.0	85.0	79.0	85.30	A	LULUS
21	MHS022	Vina Andriani	Teknik Informatika	76.0	63.0	84.0	75.30	B	LULUS
22	MHS023	Wahyu Hidayat	Sistem Informasi	92.0	86.0	74.0	83.00	B	LULUS
23	MHS024	Xena Oktavia	Ilmu Komputer	87.0	71.0	93.0	84.60	B	LULUS
24	MHS025	Yusuf Hidayah	Teknik Informatika	62.0	83.0	69.0	71.10	C	LULUS
25	MHS026	Zahra Nabila	Sistem Informasi	96.0	89.0	91.0	91.90	A	LULUS
26	MHS027	Andi Kurniawan	Ilmu Komputer	71.0	66.0	82.0	73.90	C	LULUS
27	MHS028	Bella Safira	Teknik Informatika	68.0	74.0	77.0	73.40	C	LULUS
28	MHS029	Chandra Gumilar	Sistem Informasi	79.0	70.0	94.0	82.30	B	LULUS
29	MHS030	Dewi Nurhaliza	Ilmu Komputer	64.0	78.0	99.0	82.20	B	LULUS
30	MHS031	Bryan	Sistem Informasi	89.0	99.0	100.0	96.40	A	LULUS
31	MHS032	Reyhan	Teknik Informatika	100.0	89.0	89.0	92.30	A	LULUS
32	MHS033	Jacob Mamoa	Sistem Informasi	78.0	88.0	92.0	86.60	A	LULUS
33	MHS034	Vladimir Putin	Sistem Informasi	99.0	99.0	99.0	99.00	A	LULUS
34	MHS035	Tom Halland	Teknik Informatika	78.0	69.0	89.0	79.70	B	LULUS
35	MHS036	Josh Buck	Teknik Informatika	56.0	78.0	67.0	67.00	C	LULUS

Gambar 5.1.10 Tabel sesuai Sort by ID

5.1.11 Skenario 11: *Sorting By Nama*

Fitur *sorting by nama* memberikan tabel yang berisi urutan nama mahasiswa secara alfabet (A ke Z), artinya data akan diurutkan mulai dari huruf pertama yakni a sampai z.

Masukkan angka sesuai menu yang dipilih
Pilih menu: 9

SISTEM NILAI MAHASISWA DATA AKTIF									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
1	MHS027	Andi Kurniawan	Ilmu Komputer	71.0	66.0	82.0	73.90	C	LULUS
2	MHS028	Bella Safira	Teknik Informatika	68.0	74.0	77.0	73.40	C	LULUS
3	MHS031	Bryan	Sistem Informasi	89.0	99.0	100.0	96.40	A	LULUS
4	MHS002	Budi Santoso	Sistem Informasi	88.0	73.0	95.0	86.30	A	LULUS
5	MHS029	Chandra Gumilar	Sistem Informasi	79.0	70.0	94.0	82.30	B	LULUS
6	MHS003	Citra Dewi	Ilmu Komputer	67.0	81.0	76.0	74.80	C	LULUS
7	MHS030	Dewi Nurhaliza	Ilmu Komputer	64.0	78.0	99.0	82.20	B	LULUS
8	MHS004	Dian Purnama	Teknik Informatika	90.0	64.0	71.0	74.60	C	LULUS
9	MHS005	Eko Prasetyo	Sistem Informasi	74.0	89.0	68.0	76.10	B	LULUS
10	MHS006	Fitri Handayani	Ilmu Komputer	96.0	87.0	93.0	92.10	A	LULUS
11	MHS007	Gilang Ramadhan	Teknik Informatika	72.0	84.0	79.0	78.40	B	LULUS
12	MHS008	Hana Safitri	Sistem Informasi	65.0	77.0	83.0	75.80	B	LULUS
13	MHS009	Irfan Maulana	Ilmu Komputer	52.0	57.0	59.0	56.30	D	TIDAK LULUS
14	MHS033	Jacob Mamo	Sistem Informasi	78.0	88.0	92.0	86.60	A	LULUS
15	MHS010	Joko Widodo	Teknik Informatika	91.0	88.0	80.0	85.70	A	LULUS
16	MHS036	Josh Buck	Teknik Informatika	56.0	78.0	67.0	67.00	C	LULUS
17	MHS011	Kartika Sari	Sistem Informasi	63.0	74.0	67.0	67.90	C	LULUS
18	MHS012	Luthfi Hakim	Ilmu Komputer	86.0	79.0	72.0	78.30	B	LULUS
19	MHS013	Maya Anggraini	Teknik Informatika	98.0	94.0	97.0	96.40	A	LULUS
20	MHS014	Nanda Permata	Sistem Informasi	70.0	82.0	76.0	76.00	B	LULUS
21	MHS015	Oka Mahendra	Ilmu Komputer	84.0	68.0	91.0	82.00	B	LULUS
22	MHS016	Putri Rahayu	Teknik Informatika	73.0	61.0	88.0	75.40	B	LULUS
23	MHS017	Qodir Abdillah	Sistem Informasi	69.0	75.0	90.0	79.20	B	LULUS
24	MHS032	Reyhan	Teknik Informatika	100.0	89.0	89.0	92.30	A	LULUS
25	MHS018	Rina Marlina	Ilmu Komputer	95.0	97.0	92.0	94.40	A	LULUS
26	MHS019	Sari Indah	Teknik Informatika	89.0	78.0	66.0	76.50	B	LULUS
27	MHS020	Teguh Prasetya	Sistem Informasi	81.0	72.0	87.0	80.70	B	LULUS
28	MHS035	Tom Halland	Teknik Informatika	78.0	69.0	89.0	79.70	B	LULUS
29	MHS021	Umi Kalsum	Ilmu Komputer	94.0	85.0	79.0	85.30	A	LULUS
30	MHS022	Vina Andriani	Teknik Informatika	76.0	63.0	84.0	75.30	B	LULUS
31	MHS034	Vladimir Putin	Sistem Informasi	99.0	99.0	99.0	99.00	A	LULUS
32	MHS023	Wahyu Hidayat	Sistem Informasi	92.0	86.0	74.0	83.00	B	LULUS
33	MHS024	Xena Oktavia	Ilmu Komputer	87.0	71.0	93.0	84.60	B	LULUS
34	MHS025	Yusuf Hidayah	Teknik Informatika	62.0	83.0	69.0	71.10	C	LULUS
35	MHS026	Zahra Nabila	Sistem Informasi	96.0	89.0	91.0	91.90	A	LULUS

Gambar 5.1.11 Tabel berdasarkan nama

5.1.12 Skenario 12: *Sorting By Nilai*

Fitur *sorting by nilai* memberikan tabel yang berisi urutan data berdasarkan nilai akhir secara *descending* (menurun), artinya data akan diurutkan dari nilai tertinggi hingga nilai terendah.

Pilih menu: 10

SISTEM NILAI MAHASISWA DATA AKTIF									
No	ID	Nama	Program Studi	Tugas	UTS	UAS	Akhir	Grade	Status
1	MHS034	Vladimir Putin	Sistem Informasi	99.0	99.0	99.0	99.00	A	LULUS
2	MHS031	Bryan	Sistem Informasi	89.0	99.0	100.0	96.40	A	LULUS
3	MHS013	Maya Anggraini	Teknik Informatika	98.0	94.0	97.0	96.40	A	LULUS
4	MHS018	Rina Marlina	Ilmu Komputer	95.0	97.0	92.0	94.40	A	LULUS
5	MHS032	Reyhan	Teknik Informatika	100.0	89.0	89.0	92.30	A	LULUS
6	MHS006	Fitri Handayani	Ilmu Komputer	96.0	87.0	93.0	92.10	A	LULUS
7	MHS026	Zahra Nabila	Sistem Informasi	96.0	89.0	91.0	91.90	A	LULUS
8	MHS033	Jacob Mamoa	Sistem Informasi	78.0	88.0	92.0	86.60	A	LULUS
9	MHS002	Budi Santoso	Sistem Informasi	88.0	73.0	95.0	86.30	A	LULUS
10	MHS010	Joko Widodo	Teknik Informatika	91.0	88.0	80.0	85.70	A	LULUS
11	MHS021	Umi Kalsum	Ilmu Komputer	94.0	85.0	79.0	85.30	A	LULUS
12	MHS024	Xena Oktavia	Ilmu Komputer	87.0	71.0	93.0	84.60	B	LULUS
13	MHS023	Wahyu Hidayat	Sistem Informasi	92.0	86.0	74.0	83.00	B	LULUS
14	MHS029	Chandra Gumilar	Sistem Informasi	79.0	70.0	94.0	82.30	B	LULUS
15	MHS030	Dewi Nurhaliza	Ilmu Komputer	64.0	78.0	99.0	82.20	B	LULUS
16	MHS015	Oka Mahendra	Ilmu Komputer	84.0	68.0	91.0	82.00	B	LULUS
17	MHS020	Teguh Prasetya	Sistem Informasi	81.0	72.0	87.0	80.70	B	LULUS
18	MHS035	Tom Halland	Teknik Informatika	78.0	69.0	89.0	79.70	B	LULUS
19	MHS017	Qodir Abdillah	Sistem Informasi	69.0	75.0	90.0	79.20	B	LULUS
20	MHS007	Gilang Ramadhan	Teknik Informatika	72.0	84.0	79.0	78.40	B	LULUS
21	MHS012	Luthfi Hakim	Ilmu Komputer	86.0	79.0	72.0	78.30	B	LULUS
22	MHS019	Sari Indah	Teknik Informatika	89.0	78.0	66.0	76.50	B	LULUS
23	MHS005	Eko Prasetyo	Sistem Informasi	74.0	89.0	68.0	76.10	B	LULUS
24	MHS014	Nanda Permata	Sistem Informasi	70.0	82.0	76.0	76.00	B	LULUS
25	MHS008	Hana Safitri	Sistem Informasi	65.0	77.0	83.0	75.80	B	LULUS
26	MHS016	Putri Rahayu	Teknik Informatika	73.0	61.0	88.0	75.40	B	LULUS
27	MHS022	Vina Andriani	Teknik Informatika	76.0	63.0	84.0	75.30	B	LULUS
28	MHS003	Citra Dewi	Ilmu Komputer	67.0	81.0	76.0	74.80	C	LULUS
29	MHS004	Dian Purnama	Teknik Informatika	90.0	64.0	71.0	74.60	C	LULUS
30	MHS027	Andi Kurniawan	Ilmu Komputer	71.0	66.0	82.0	73.90	C	LULUS
31	MHS028	Bella Safira	Teknik Informatika	68.0	74.0	77.0	73.40	C	LULUS
32	MHS025	Yusuf Hidayah	Teknik Informatika	62.0	83.0	69.0	71.10	C	LULUS
33	MHS011	Kartika Sari	Sistem Informasi	63.0	74.0	67.0	67.90	C	LULUS
34	MHS036	Josh Buck	Teknik Informatika	56.0	78.0	67.0	67.00	C	LULUS
35	MHS009	Irfan Maulana	Ilmu Komputer	52.0	57.0	59.0	56.30	D	TIDAK LULUS

Gambar 5.1.12 Tabel berdasarkan nilai

5.1.13 Skenario 13: Tabel Statistik

Fitur akan memberikan ringkasan kondisi akademik seluruh mahasiswa. Tabel berisi informasi berupa: total mahasiswa aktif, jumlah lulus, jumlah tidak lulus, rata-rata nilai akhir, persentase kelulusan, serta distribusi *grade* A hingga E.

Masukkan angka sesuai menu yang dipilih
Pilih menu: 11

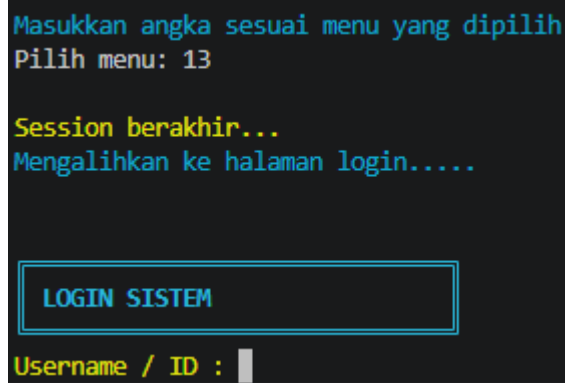
STATISTIK MAHASISWA	
Total Mahasiswa	: 35
Jumlah Lulus	: 34
Jumlah Tidak Lulus	: 1
Rata-Rata	: 80.88
Persentase Lulus	: 97.14 %

Distribusi Nilai	
A	: 11
B	: 16
C	: 7
D	: 1
E	: 0

Gambar 5.1.6.13 Tabel Statistik

5.1.14 Skenario 14: Logout dan login kembali

Fitur ini memungkinkan untuk melakukan aktivitas login kembali tanpa harus mengakhiri program terlebih dahulu. Pengguna dapat dengan tanpa ribet menjalankan program berulang kali.



Gambar 5.1.14 Tampilan logout dan login

BAB VI

PENUTUP

6.1 Kesimpulan

Pengembangan Sistem Manajemen Nilai Mahasiswa berbasis Java dalam praktikum ini membuktikan bahwa konsep algoritma dan struktur data dapat diimplementasikan secara langsung untuk menyelesaikan permasalahan pengelolaan data akademik secara efektif dan efisien. Pertama, sistem berhasil dibangun dengan memanfaatkan struktur data array bertipe objek Mahasiswa[] sebagai fondasi penyimpanan data yang mendukung seluruh proses manipulasi data mulai dari penambahan, pengeditan, hingga penghapusan secara terstruktur. Kedua, implementasi algoritma *Sequential Search* pada pencarian nama dan program studi serta *Binary Search* pada pencarian ID membuktikan bahwa pemilihan algoritma yang tepat berpengaruh signifikan terhadap efisiensi sistem, di mana *Binary Search* mampu mereduksi kompleksitas pencarian dari $O(n)$ menjadi $O(\log n)$. Ketiga, penggunaan algoritma *Quick Sort* dengan pendekatan *divide and conquer* terbukti efektif dalam mengurutkan data berdasarkan ID, nama, maupun nilai akhir dengan kompleksitas rata-rata $O(n \log n)$, menjadikannya lebih unggul dibanding algoritma pengurutan sederhana. Keempat, penerapan konsep *soft delete* pada fitur hapus dan restore data berhasil mengatasi risiko kehilangan data secara permanen sekaligus memberikan fleksibilitas pengelolaan data bagi administrator. Kelima, mekanisme *auto save* menggunakan file eksternal data.txt memastikan keberlangsungan data meskipun program dihentikan, sehingga sistem memiliki persistensi data yang andal tanpa bergantung pada database. Keenam, analisis kompleksitas yang dilakukan menunjukkan bahwa sistem secara keseluruhan beroperasi pada rentang efisiensi $O(1)$ hingga $O(n \log n)$, yang mengindikasikan bahwa kombinasi algoritma yang digunakan sudah cukup optimal untuk skala data praktikum.

6.2 Saran

Sistem Manajemen Nilai Mahasiswa yang telah dikembangkan masih memiliki sejumlah keterbatasan yang dapat dijadikan bahan perbaikan dan pengembangan lebih lanjut.

- a. Pertama, kapasitas penyimpanan data yang saat ini dibatasi pada 200 mahasiswa menggunakan array statis sebaiknya digantikan dengan struktur data dinamis seperti ArrayList atau LinkedList agar sistem mampu menangani jumlah data yang lebih besar tanpa batasan kapasitas tetap.
- b. Kedua, mekanisme penyimpanan data yang masih menggunakan file teks biasa data.txt sebaiknya ditingkatkan dengan mengintegrasikan sistem basis data seperti MySQL atau SQLite sehingga pengelolaan data menjadi lebih aman, terstruktur, dan skalabel.
- c. Ketiga, proses pencarian nama dan program studi yang saat ini masih menggunakan *Sequential Search* dengan kompleksitas $O(n)$ sebaiknya dioptimalkan menggunakan struktur data HashMap atau teknik *indexing* agar waktu pencarian dapat direduksi secara signifikan ketika jumlah data bertambah besar.
- d. Keempat, antarmuka sistem yang masih berbasis *console* sebaiknya dikembangkan menjadi antarmuka grafis (*Graphical User Interface/GUI*) menggunakan JavaFX atau framework lainnya agar sistem lebih mudah digunakan oleh pengguna umum yang tidak familiar dengan perintah terminal.
- e. Kelima, fitur validasi input pada sistem perlu diperkuat dengan menambahkan mekanisme penanganan kesalahan (*exception handling*) yang lebih komprehensif sehingga program tidak mudah berhenti secara tidak terduga akibat input yang tidak sesuai.
- f. Keenam, sistem ke depannya dapat dikembangkan dengan menambahkan fitur ekspor data ke format yang lebih umum seperti PDF atau Excel agar laporan nilai mahasiswa dapat didistribusikan dan diarsipkan dengan lebih praktis.